



AES and SM4

Step-by-Step Encryption, Differential Fault Analysis, and Key Recovery

Educational Technical Notes for PFE 5A

Student: Guilherme Specht

Supervisor: Prof. Philippe Maurine

Institution: Polytech Montpellier

Industrial context: STMicroelectronics

Date: 7 November 2026

Complete report. All explanatory chapters are included. This report explains the functions, logic, and structures; the full source listings are kept as separate appendix builds (`part_06`–`part_11`).

Contents

1 Purpose and Notation **1**

2 AES-128: Encryption from the Beginning **2**
2.1 Block, key, and round structure . . . 2
2.2 Initial AddRoundKey . . . 2
2.3 SubBytes . . . 2
2.4 ShiftRows . . . 3
2.5 MixColumns . . . 3
2.6 Round keys . . . 3

3 Piret–Quisquater Differential Fault Analysis of AES **4**
3.1 Where the fault is injected . . . 4
3.2 Propagation through MixColumns . . . 4
3.3 Which ciphertext bytes are affected? . . . 4
3.4 Constructing candidates for the last AES round key . . . 5
3.5 Intersecting observations . . . 5
3.6 Reversing AES-128 key expansion . . . 5

4 SM4: Complete Structure and First Round **6**
4.1 Official test vector . . . 6
4.2 SM4 round function . . . 6
4.3 SM4 key expansion . . . 6
4.4 Detailed computation of encryption round 0 . . . 7

5 Differential Fault Analysis of SM4: A Worked Last-Round Model **8**
5.1 Attack model and limitations . . . 8
5.2 Known ciphertext and hidden internal values . . . 8
5.3 Step 1: Inject a known XOR fault . . . 8
5.4 Step 2: Observe the S-box difference . . . 8
5.5 Step 3: Propagate the difference through L . . . 9
5.6 Step 4: Undo the linear layer from observed data . . . 9
5.7 Step 5: Convert S-box input candidates to key candidates . . . 9
5.8 Step 6: Use independent observations . . . 9
5.9 Step 7: Repeat for the other three bytes . . . 10
5.10 What happens after recovering the last subkey? . . . 10

6 SM4: Recovering the Master Key from Four Round Keys **11**
6.1 Required subkeys . . . 11
6.2 Reverse the recurrence . . . 11
6.3 Undo the initial constants . . . 11

7 Giraud Fault Attack on AES-128 and Its Adaptation to SM4 **12**
7.1 Core idea and comparison with Piret–Quisquater . . . 12
7.2 AES-128: precise fault location . . . 12
7.3 Worked AES-128 example: the first ciphertext byte . . . 12
7.4 Multiple faults and candidate intersection . . . 13
7.5 Adaptation of the single-bit idea to SM4 . . . 13
7.6 Worked SM4 example and recovery of rk_{31} . . . 14
7.7 From the final SM4 subkey to the master key . . . 15
7.8 Experimental implications . . . 15

8 AES versus SM4: Key Lessons **16**

9 Python Implementation and Reproducible Terminal Results **17**
9.1 Repository structure and execution . . . 17
9.2 AES-128 implementation: S-box, diffusion, and key expansion . . . 17
9.3 SM4 implementation: rotations, S-boxes, and 32 rounds . . . 19
9.4 AES Giraud: filtering last-round key candidates . . . 20
9.5 SM4 Giraud-inspired filtering . . . 21
9.6 Recovering SM4 rounds 31 through 28 . . . 23
9.7 Reversing the SM4 key schedule . . . 24
9.8 Interpretation of the results . . . 25

10 Complete Code Walkthrough: Every Implemented Attack **26**
10.1 Which attacks are implemented? . . . 26
10.2 Normal cipher, simulated target, solver, and viewer . . . 27
10.3 AES Piret-style attack: code from injection to the master key . . . 27
10.4 AES Giraud attack: one-bit filtering and complete recovery . . . 30
10.5 SM4 controlled-byte DFA: code and full-key extension . . . 30
10.6 SM4 Giraud-inspired attack: reuse with a different filter . . . 32
10.7 How the GUI maps to the attack code . . . 32
10.8 Commands and verified default examples . . . 33
10.9 Validation and limits of the implementation . . . 33

11 Implementation Manual: Every Function and Its Execution Order **34**
11.1 Reading Python data and control flow . . . 34
11.2 Module-level configuration and initialization . . . 34
11.3 End-to-end trace: ordinary AES and SM4 . . . 35
11.4 End-to-end trace: the four fault-recovery paths . . . 36
11.5 How to use the function reference . . . 36
11.6 `aes.py`: function-by-function implementation . . . 36
11.7 `sm4.py`: function-by-function implementation . . . 41

12 Attack Functions: Piret and Giraud **45**
12.1 `piret.py`: function-by-function implementation . . . 45
12.2 `giraud.py`: function-by-function implementation . . . 53

13 Terminal and GUI Functions **57**
13.1 `run.py`: function-by-function implementation . . . 57
13.2 `gui.py`: function-by-function implementation . . . 60
13.3 Return values, mutation and error handling at a glance . . . 68

References and Scope **69**

Chapter 1

Purpose and Notation

This document reorganizes the supplied handwritten AES/SM4 notes into a progressive English explanation. It distinguishes established algorithmic facts, pedagogical fault simulations, and the additional work needed for a physical attack.

- $\oplus$: bitwise XOR; $\lll n$: 32-bit circular left rotation by n bits.
- S: an 8-bit substitution box (AES and SM4 use different S-boxes).
- C: correct ciphertext; F: faulty ciphertext; $\Delta = C \oplus F$.
- All hexadecimal words are written most-significant byte first. SM4 words are 32 bits.

Note. A known plaintext/key test vector is used to *verify* examples. In an actual key-recovery experiment, the attacker does not know the secret key.

Chapter 2

AES-128: Encryption from the Beginning

2.1 Block, key, and round structure

AES is a 128-bit block cipher. AES-128, AES-192, and AES-256 use 10, 12, and 14 rounds respectively. We focus on AES-128.

Plaintext	`00112233445566778899AABBCCDDEEFF`
Key K_0	`000102030405060708090A0B0C0D0E0F`
Ciphertext	`69C4E0D86A7B0430D8CDB78070B4C55A`

AES stores bytes in a 4×4 *column-major* state:

$$\begin{pmatrix} 00 & 44 & 88 & CC \\ 11 & 55 & 99 & DD \\ 22 & 66 & AA & EE \\ 33 & 77 & BB & FF \end{pmatrix}.$$

First apply AddRoundKey with K_0. Rounds 1–9 apply SubBytes, ShiftRows, MixColumns, AddRoundKey. Round 10 omits MixColumns.

2.2 Initial AddRoundKey

The state is XORed byte by byte with the first round key. For example, $\texttt{11} \oplus \texttt{01} = \texttt{10}$. The result is

$$\begin{pmatrix} 00 & 40 & 80 & C0 \\ 10 & 50 & 90 & D0 \\ 20 & 60 & A0 & E0 \\ 30 & 70 & B0 & F0 \end{pmatrix}.$$

2.3 SubBytes

Each byte is independently replaced through the AES S-box. Examples: $S(\texttt{00}) = \texttt{63}$, $S(\texttt{10}) = \texttt{CA}$, $S(\texttt{20}) = \texttt{B7}$, $S(\texttt{30}) = \texttt{04}$. The resulting state is

$$\begin{pmatrix} 63 & 09 & CD & BA \\ CA & 53 & 60 & 70 \\ B7 & D0 & E0 & E1 \\ 04 & 51 & E7 & 8C \end{pmatrix}.$$

The S-box is nonlinear: in general, $S(a \oplus b) \neq S(a) \oplus S(b)$.

2.4 ShiftRows

Rows 0, 1, 2, 3 rotate left by 0, 1, 2, 3 bytes respectively:

$$\begin{pmatrix} 63 & 09 & CD & BA \\ 53 & 60 & 70 & CA \\ E0 & E1 & B7 & D0 \\ 8C & 04 & 51 & E7 \end{pmatrix}.$$

2.5 MixColumns

Every column is multiplied over $GF(2^8)$ by

$$M = \begin{pmatrix} 02 & 03 & 01 & 01 \\ 01 & 02 & 03 & 01 \\ 01 & 01 & 02 & 03 \\ 03 & 01 & 01 & 02 \end{pmatrix}.$$

For the first input column $(\texttt{63}, \texttt{53}, \texttt{E0}, \texttt{8C})^T$,

$$b_0 = (02 \cdot 63) \oplus (03 \cdot 53) \oplus E0 \oplus 8C.$$

Multiplication is in the AES finite field, not ordinary integer arithmetic. For a separate illustrative column $(\texttt{DB}, \texttt{13}, \texttt{53}, \texttt{45})^T$, MixColumns yields $(\texttt{8E}, \texttt{4D}, \texttt{A1}, \texttt{BC})^T$. The first column of the official AES round-1 state becomes $(\texttt{5F}, \texttt{72}, \texttt{64}, \texttt{15})^T$.

2.6 Round keys

AES-128 expands the original 16-byte key into eleven 16-byte round keys $K_0, \ldots, K_{10}$. For this vector:

$$K_1 = \texttt{D6AA74FDD2AF72FADAA678F1D6AB76FE},$$

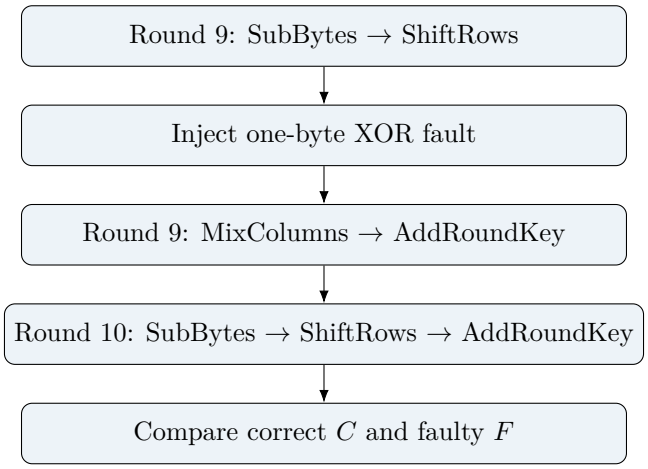
$$K_{10} = \texttt{13111D7FE3944A17F307A78B4D2B30C5}.$$

Chapter 3

Piret–Quisquater Differential Fault Analysis of AES

3.1 Where the fault is injected

In the classic model used in the notes, one nonzero byte is corrupted at the *input of MixColumns in round 9*, after SubBytes and ShiftRows. Both correct and faulty executions then proceed through round 9 and the final round.



3.2 Propagation through MixColumns

If the first byte of a column changes by e, the column difference after MixColumns is

$$\Delta_{MC} = (02e,\ e,\ e,\ 03e)^T.$$

For $e = \texttt{01}$, this is $(\texttt{02}, \texttt{01}, \texttt{01}, \texttt{03})^T$. AddRoundKey does not alter XOR differences because both executions use the same round key. The final SubBytes transforms these differences nonlinearly; ShiftRows moves them into separate ciphertext positions.

3.3 Which ciphertext bytes are affected?

For the four round-9 columns, the affected ciphertext positions (in corresponding row order) are:

Faulted column	Ciphertext byte positions
0	0, 13, 10, 7
1	4, 1, 14, 11
2	8, 5, 2, 15
3	12, 9, 6, 3

3.4 Constructing candidates for the last AES round key

For an affected ciphertext position j, and a guessed last-round key byte k, compute

$$d_j(k) = S^{-1}(C_j \oplus k) \oplus S^{-1}(F_j \oplus k).$$

Each byte has 256 candidate values. Four unknown key bytes have $256^4 = 2^{32}$ possible combinations before filtering. Construct a set $\mathcal{D}$ containing the MixColumns output differences for a single nonzero faulty byte in any of the four row positions:

$$\mathcal{D} = \{M(e\mathbf{e}_r) : r \in \{0, 1, 2, 3\},\ e \in \{1, \ldots, 255\}\}.$$

This produces $4 \cdot 255 = 1020$ four-byte difference patterns (not 1020 individual bytes). For each proposed key quartet, compute the four $d_j(k)$ values in their correct *pre-ShiftRows row order* and keep the quartet only if the resulting vector lies in $\mathcal{D}$.

3.5 Intersecting observations

A single correct/faulty pair typically leaves multiple candidate key quartets. Repeat with independent faults in the same column and intersect the candidate sets:

$$\mathcal{K}_{\text{surviving}} = \mathcal{K}_1 \cap \mathcal{K}_2 \cap \cdots .$$

Repeat across all four columns to recover the full 16-byte K_{10}. The notes illustrate candidate filtering using a hypothetical quartet $(\texttt{13}, \texttt{2B}, \texttt{A7}, \texttt{17})$; such pedagogical candidate examples should not be confused with a complete independently verified AES fault trace.

3.6 Reversing AES-128 key expansion

Let $w_0, \ldots, w_{43}$ be 32-bit expanded-key words. For $i \geq 4$,

$$w_i = \begin{cases} w_{i-4} \oplus g(w_{i-1}), & i \equiv 0 \pmod 4, \\ w_{i-4} \oplus w_{i-1}, & \text{otherwise}, \end{cases}$$

where $g(w) = \text{SubWord}(\text{RotWord}(w)) \oplus \text{Rcon}$. In reverse, compute w_{i-4} by XORing w_i with the same known right-hand transformation. Begin with $w_{40}, \ldots, w_{43}$ from K_{10} and work backward to $w_0, \ldots, w_3$, yielding K_0. No inverse S-box is required for this reversal because the relevant inputs to g are already known.

Chapter 4

SM4: Complete Structure and First Round

4.1 Official test vector

Plaintext	`0123456789ABCDEFFEDCBA9876543210`
Master key	`0123456789ABCDEFFEDCBA9876543210`
Ciphertext	`681EDF34D206965E86B3E94F536E4246`

Split the plaintext into four 32-bit words:

$$X_0 = \texttt{01234567}, \quad X_1 = \texttt{89ABCDEF}, \quad X_2 = \texttt{FEDCBA98}, \quad X_3 = \texttt{76543210}.$$

4.2 SM4 round function

SM4 performs 32 rounds, producing one new 32-bit word per round:

$$\boxed{X_{i+4} = X_i \oplus T(X_{i+1} \oplus X_{i+2} \oplus X_{i+3} \oplus rk_i)}.$$

Write $T = L \circ \tau$, where τ substitutes the four bytes through the SM4 S-box and

$$L(B) = B \oplus (B \lll 2) \oplus (B \lll 10) \oplus (B \lll 18) \oplus (B \lll 24).$$

The output reverses the final four words:

$$C = X_{35} \parallel X_{34} \parallel X_{33} \parallel X_{32}.$$

4.3 SM4 key expansion

The four initial key-schedule words are $K_i = MK_i \oplus FK_i$, with

$$FK_0 = \texttt{A3B1BAC6}, \quad FK_1 = \texttt{56AA3350},$$
$$FK_2 = \texttt{677D9197}, \quad FK_3 = \texttt{B27022DC}.$$

The key-schedule recurrence is

$$K_{i+4} = K_i \oplus T'(K_{i+1} \oplus K_{i+2} \oplus K_{i+3} \oplus CK_i), \quad rk_i = K_{i+4},$$

with $T' = L' \circ \tau$ and

$$L'(B) = B \oplus (B \lll 13) \oplus (B \lll 23).$$

Crucial distinction: encryption uses L, while key expansion uses L'. They have different rotation constants. For the first round, $rk_0 = \texttt{F12186F9}$.

4.4 Detailed computation of encryption round 0

Step 1: XOR the three newest words with the round key.

$$A = X_1 \oplus X_2 \oplus X_3 \oplus rk_0 = \texttt{F002C39E}.$$

Step 2: Apply the SM4 S-box to each byte.

$$\texttt{F0} \mapsto \texttt{18}, \quad \texttt{02} \mapsto \texttt{E9}, \quad \texttt{C3} \mapsto \texttt{92}, \quad \texttt{9E} \mapsto \texttt{B1}.$$

Thus $B = \tau(A) = \texttt{18E992B1}$.

Step 3: Apply the linear transformation.

Term	Hexadecimal value
B	18E992B1
$B \lll 2$	63A64AC4
$B \lll 10$	A64AC463
$B \lll 18$	4AC463A6
$B \lll 24$	B118E992
XOR of all five	26D99622

Step 4: XOR with the oldest word.

$$X_4 = X_0 \oplus L(B) = \texttt{01234567} \oplus \texttt{26D99622} = \boxed{\texttt{27FAD345}}.$$

The next four-word window is (X_1, X_2, X_3, X_4). The following words are $X_5 = \texttt{A18B4CB2}$, $X_6 = \texttt{11C1E22A}$, and $X_7 = \texttt{CC13E2EE}$. After 32 rounds and the output reversal, the official ciphertext is obtained.

Chapter 5

Differential Fault Analysis of SM4: A Worked Last-Round Model

5.1 Attack model and limitations

We use a *controlled, known XOR fault* in one byte of the input to the SM4 S-boxes in round 31. This is a pedagogical model, not a claim that a real clock/voltage glitch yields a known XOR value. In this model, all other round inputs remain unchanged. The last round is

$$X_{35} = X_{31} \oplus L(\tau(A)), \qquad A = X_{32} \oplus X_{33} \oplus X_{34} \oplus rk_{31}.$$

The attacker observes the correct ciphertext and a faulty ciphertext. The secret internal values below are supplied *only for checking the simulation.*

5.2 Known ciphertext and hidden internal values

$$X_{35} = \texttt{681EDF34}, \quad X_{34} = \texttt{D206965E},$$
$$X_{33} = \texttt{86B3E94F}, \quad X_{32} = \texttt{536E4246}.$$

For this official test vector, the simulation has

$$X_{31} = \texttt{7B938F4C}, \quad A = \texttt{96FF9D45}, \quad rk_{31} = \texttt{9124A012}.$$

The attacker must recover the key rather than assume it.

5.3 Step 1: Inject a known XOR fault

Alter the first byte of A by $e = \texttt{01}$:

$$A = \texttt{96FF9D45} \longrightarrow A' = \texttt{97FF9D45}.$$

5.4 Step 2: Observe the S-box difference

The two S-box outputs are

$$B = \tau(A) = \texttt{1A488C73}, \qquad B' = \tau(A') = \texttt{55488C73}.$$

Consequently,

$$\Delta B = B \oplus B' = \texttt{4F000000}.$$

5.5 Step 3: Propagate the difference through L

Because L is XOR-linear,

$$\Delta X_{35} = L(B) \oplus L(B') = L(\Delta B) = \texttt{734E3D3D}.$$

The correct last word is $X_{35} = \texttt{681EDF34}$, so the faulty last word is

$$X'_{35} = \texttt{681EDF34} \oplus \texttt{734E3D3D} = \texttt{1B50E209}.$$

Therefore the attacker sees

Correct ciphertext	681EDF34	D206965E	86B3E94F	536E4246
Faulty ciphertext	1B50E209	D206965E	86B3E94F	536E4246
Difference	734E3D3D	00000000	00000000	00000000

5.6 Step 4: Undo the linear layer from observed data

The attacker computes

$$\Delta B = L^{-1}(\Delta X_{35}) = L^{-1}(\texttt{734E3D3D}) = \texttt{4F000000}.$$

The first S-box byte therefore satisfies

$$S(a) \oplus S(a \oplus \texttt{01}) = \texttt{4F}.$$

Enumerating all 256 possible input bytes gives the candidate set

$$\mathcal{A}_1 = \{\texttt{96}, \texttt{97}\}.$$

5.7 Step 5: Convert S-box input candidates to key candidates

The attacker knows the last three words from the ciphertext:

$$X_{32} \oplus X_{33} \oplus X_{34} = \texttt{07DB3D57}.$$

Since $rk_{31} = A \oplus \texttt{07DB3D57}$, the first byte of the key is

$$rk_{31}[0] = a \oplus \texttt{07}.$$

Thus $a = \texttt{96}$ yields $\texttt{91}$, while $a = \texttt{97}$ yields $\texttt{90}$. After one observation, the candidates are $\{\texttt{91}, \texttt{90}\}$.

5.8 Step 6: Use independent observations

Repeat with the *same plaintext and state*, but use a known XOR fault $e = \texttt{02}$. The new S-box difference is $d_2 = \texttt{81}$. Keep only input candidates satisfying both

$$\begin{cases} S(a) \oplus S(a \oplus \texttt{01}) = \texttt{4F}, \\ S(a) \oplus S(a \oplus \texttt{02}) = \texttt{81}. \end{cases}$$

The intersection identifies $a = \texttt{96}$, hence $rk_{31}[0] = \texttt{91}$.

5.9 Step 7: Repeat for the other three bytes

The supplied worked example reports the following bytewise observations under the same controlled-fault assumption:

Byte	d for $e = 01$	d for $e = 02$	Recovered A	Recovered rk_{31}
0	4F	81	96	91
1	71	83	FF	24
2	79	6F	9D	A0
3	80	C9	45	12

This reconstructs

$$A = \texttt{96FF9D45}, \qquad rk_{31} = A \oplus \texttt{07DB3D57} = \boxed{\texttt{9124A012}}.$$

Note. This illustrates last-round subkey recovery *conditional on the fault model*. The listed observations should be validated against a software implementation before treating them as experimental results.

5.10 What happens after recovering the last subkey?

Knowing rk_{31}, reconstruct the previous state word:

$$X_{31} = X_{35} \oplus T(X_{32} \oplus X_{33} \oplus X_{34} \oplus rk_{31}).$$

For the example, $X_{31} = \texttt{7B938F4C}$. However, this does *not* automatically reveal rk_{30}. Additional fault observations and state constraints are required for rounds 30, 29, and 28. The present worked example does not demonstrate that stage of the attack.

Chapter 6

SM4: Recovering the Master Key from Four Round Keys

6.1 Required subkeys

If four consecutive final subkeys have been recovered, then

Key-schedule word	Round key
K_{32}	$rk_{28} =$ `428D3654`
K_{33}	$rk_{29} =$ `62293496`
K_{34}	$rk_{30} =$ `01CF72E5`
K_{35}	$rk_{31} =$ `9124A012`

These are reference subkeys from the official example; *only* rk_{31} was recovered in the preceding pedagogical DFA walkthrough. the attack walkthrough in Part 2 explains how the current code actually recovers all four subkeys with a selectable-round fault oracle.

6.2 Reverse the recurrence

The forward key schedule is

$$K_{i+4} = K_i \oplus T'(K_{i+1} \oplus K_{i+2} \oplus K_{i+3} \oplus CK_i).$$

XOR is self-inverse, so

$$\boxed{K_i = K_{i+4} \oplus T'(K_{i+1} \oplus K_{i+2} \oplus K_{i+3} \oplus CK_i)}.$$

Start with $i = 31$, then decrease to $i = 0$. For example,

$$K_{31} = K_{35} \oplus T'(K_{32} \oplus K_{33} \oplus K_{34} \oplus CK_{31}),$$

where $CK_{31} =$ `646B7279`. This yields the previous expanded-key word $K_{31} =$ `F1780C81` in the official test vector. Continue to recover K_0, K_1, K_2, K_3.

6.3 Undo the initial constants

Finally,

$$MK_i = K_i \oplus FK_i \qquad (i = 0, 1, 2, 3),$$

and concatenate the four master-key words:

$$\boxed{MK = \texttt{0123456789ABCDEFFEDCBA9876543210}}.$$

Chapter 7

Giraud Fault Attack on AES-128 and Its Adaptation to SM4

7.1 Core idea and comparison with Piret–Quisquater

Giraud's AES fault-analysis approach exploits a *single-bit* fault close to the last round. In contrast to the Piret–Quisquater example, which injects a byte fault before the ninth-round MixColumns and typically modifies four ciphertext bytes, a single-byte fault immediately before the final SubBytes affects only one ciphertext byte: the last AES round does not contain MixColumns. The attacker tests last-round key-byte candidates by checking whether the inverse-S-box input difference has Hamming weight one.

7.2 AES-128: precise fault location

Let U denote the state after round 9 AddRoundKey, just before round 10 SubBytes. A one-bit XOR fault modifies a single input byte a into $a' = a \oplus e$, where $e \in \{\texttt{01}, \texttt{02}, \texttt{04}, \texttt{08}, \texttt{10}, \texttt{20}, \texttt{40}, \texttt{80}\}$. Round 10 performs SubBytes, ShiftRows and AddRoundKey K_{10}; there is no MixColumns. ShiftRows relocates the changed byte without increasing the number of affected bytes.

For the affected ciphertext position j, with correct byte c, faulty byte f, and unknown last-round key byte k, the attacker evaluates

$$\delta(k) = S_{\mathrm{AES}}^{-1}(c \oplus k) \oplus S_{\mathrm{AES}}^{-1}(f \oplus k). \tag{7.1}$$

The candidate is compatible with the fault model if

$$\delta(k) \in \mathcal{E} = \{\texttt{01}, \texttt{02}, \texttt{04}, \texttt{08}, \texttt{10}, \texttt{20}, \texttt{40}, \texttt{80}\}. \tag{7.2}$$

Equivalently, the 8-bit value $\delta(k)$ must have Hamming weight exactly one. If the injected bit position is known, the condition can be made stronger: $\delta(k) = e$.

7.3 Worked AES-128 example: the first ciphertext byte

We use the official AES-128 test vector already introduced in this report:

$$P = \texttt{00112233445566778899AABBCCDDEEFF},$$
$$K_0 = \texttt{000102030405060708090A0B0C0D0E0F},$$
$$C = \texttt{69C4E0D86A7B0430D8CDB78070B4C55A}.$$

The first byte entering the final SubBytes is $a = \texttt{BD}$. Inject $e = \texttt{01}$ in that byte:

$$\texttt{BD} \oplus \texttt{01} = \texttt{BC}. \tag{7.3}$$

The AES S-box maps $S(\texttt{BD}) = \texttt{7A}$ and $S(\texttt{BC}) = \texttt{65}$. The first byte of K_{10} is $\texttt{13}$, so

$$c = \texttt{7A} \oplus \texttt{13} = \texttt{69},$$
$$f_1 = \texttt{65} \oplus \texttt{13} = \texttt{76}.$$

Thus the simulated faulty ciphertext is

$$F_1 = \texttt{76C4E0D86A7B0430D8CDB78070B4C55A}. \tag{7.4}$$

The attacker does not assume the value $\texttt{13}$: it is shown here solely to verify the example. For candidate $k = \texttt{13}$, equation (7.1) gives

$$S^{-1}(\texttt{69} \oplus \texttt{13}) = S^{-1}(\texttt{7A}) = \texttt{BD},$$
$$S^{-1}(\texttt{76} \oplus \texttt{13}) = S^{-1}(\texttt{65}) = \texttt{BC},$$
$$\delta(\texttt{13}) = \texttt{BD} \oplus \texttt{BC} = \boxed{\texttt{01}}.$$

The candidate is compatible with a single-bit fault, but one correct/faulty pair may leave several candidates.

7.4 Multiple faults and candidate intersection

Repeat the encryption with the same plaintext and key and introduce an independent single-bit fault at the same location. In the worked example, choosing $e_2 = \texttt{02}$ produces $a'_2 = \texttt{BF}$ and a second faulty first ciphertext byte $f_2 = \texttt{1B}$. For observation t, define

$$\mathcal{K}_t = \{k \in \{0, \ldots, 255\} : S^{-1}(c \oplus k) \oplus S^{-1}(f_t \oplus k) \in \mathcal{E}\}. \tag{7.5}$$

The surviving candidates are the intersection $\mathcal{K} = \bigcap_t \mathcal{K}_t$. For the illustrated pair of faults, the recovered first byte is $K_{10}[0] = \texttt{13}$. The number of observations required is not fixed: it depends on the fault model and the observed differences.

Repeating the experiment across all sixteen pre-final-SubBytes byte positions allows the attacker, under this model, to reconstruct all sixteen bytes of K_{10}. For the official vector,

$$K_{10} = \texttt{13111D7FE3944A17F307A78B4D2B30C5}. \tag{7.6}$$

The AES-128 key schedule can then be reversed to recover the original key K_0, as explained in the preceding AES key-expansion chapter.

7.5 Adaptation of the single-bit idea to SM4

The corresponding SM4 experiment is an *adaptation of the single-bit differential filtering principle*, rather than a claim that the original AES attack transfers unchanged. The SM4 last round is

$$X_{35} = X_{31} \oplus L(\tau(A)), \qquad A = X_{32} \oplus X_{33} \oplus X_{34} \oplus rk_{31}. \tag{7.7}$$

The output is $C = (X_{35}, X_{34}, X_{33}, X_{32})$. Inject a single-bit fault in one byte of A, just before the last-round S-boxes. Unlike AES, the SM4 linear map L diffuses the one-byte S-box output difference over the 32-bit word X_{35}. The other three ciphertext words remain unchanged *under the assumed isolated-fault model.*

Let F be the faulty ciphertext and $\Delta X_{35} = X_{35} \oplus X'_{35}$. Since L is linear and invertible, the attacker can compute

$$\Delta B = L^{-1}(\Delta X_{35}) = \tau(A) \oplus \tau(A'). \tag{7.8}$$

For an affected byte with S-box input a and unknown single-bit error e, candidate inputs satisfy

$$S_{\mathrm{SM4}}(a) \oplus S_{\mathrm{SM4}}(a \oplus e) = d, \quad e \in \mathcal{E}, \tag{7.9}$$

where d is the corresponding byte of ΔB. If e is known from the simulation, it is used directly; otherwise all eight one-bit masks must be tested. Multiple correct/faulty observations reduce the candidate set by intersection.

7.6 Worked SM4 example and recovery of rk_{31}

Using the official SM4 test vector,

$$\begin{aligned} P &= \texttt{0123456789ABCDEFFEDCBA9876543210}, \\ MK &= \texttt{0123456789ABCDEFFEDCBA9876543210}, \\ C &= \texttt{681EDF34D206965E86B3E94F536E4246}, \end{aligned}$$

the simulated input of the last S-box layer is $A = \texttt{96FF9D45}$. A one-bit fault $e = \texttt{01000000}$ gives $A' = \texttt{97FF9D45}$. The substitution outputs are

$$\begin{aligned} \tau(A) &= \texttt{1A488C73}, \\ \tau(A') &= \texttt{55488C73}, \\ \Delta B &= \texttt{4F000000}. \end{aligned}$$

After the linear layer,

$$\Delta X_{35} = L(\texttt{4F000000}) = \texttt{734E3D3D}. \tag{7.10}$$

Therefore the faulty output is

$$F_1 = \texttt{1B50E209D206965E86B3E94F536E4246}. \tag{7.11}$$

From the observed difference alone, the attacker obtains $\Delta B = L^{-1}(\texttt{734E3D3D}) = \texttt{4F000000}$. With known fault mask `01` in the first byte, the candidate equation is

$$S_{\mathrm{SM4}}(a) \oplus S_{\mathrm{SM4}}(a \oplus \texttt{01}) = \texttt{4F}. \tag{7.12}$$

The first observation permits $a \in \{\texttt{96}, \texttt{97}\}$. A second simulated fault of mask `02` gives the first-byte S-box difference `81`. Intersecting both sets identifies $a = \texttt{96}$.

The three known ciphertext words yield

$$X_{32} \oplus X_{33} \oplus X_{34} = \texttt{07DB3D57}. \tag{7.13}$$

Thus the first last-round key byte is

$$rk_{31}[0] = \texttt{96} \oplus \texttt{07} = \boxed{\texttt{91}}. \tag{7.14}$$

The supplied worked example applies analogous observations to the remaining three S-box inputs, obtaining

$$A = \texttt{96FF9D45}, \qquad rk_{31} = A \oplus \texttt{07DB3D57} = \boxed{\texttt{9124A012}}. \tag{7.15}$$

The bytewise simulated differences are summarized below.

Byte	d for $e = 01$	d for $e = 02$	Recovered A	Recovered rk_{31}
0	4F	81	96	91
1	71	83	FF	24
2	79	6F	9D	A0
3	80	C9	45	12

Note. The numerical values in this subsection are simulated, not physical fault-injection measurements. In particular, knowledge of the exact XOR mask is a strong experimental assumption.

7.7 From the final SM4 subkey to the master key

Recovering rk_{31} is not equivalent to recovering the 128-bit master key. The SM4 key schedule needs four consecutive 32-bit round keys to be inverted. In the official test vector these are

$$rk_{28} = \texttt{428D3654}, \qquad rk_{29} = \texttt{62293496},$$
$$rk_{30} = \texttt{01CF72E5}, \qquad rk_{31} = \texttt{9124A012}.$$

The values rk_{28}, rk_{29} and rk_{30} are *reference values*, not keys shown to have been recovered by the two last-round faults. The two last-round faults do not recover these other keys. The implemented multi-round extension in the attack walkthrough in Part 2 obtains them by requesting new faults in rounds 30, 29 and 28 and undoing previously recovered later rounds.

With four consecutive round keys, set $K_{i+4} = rk_i$ and invert the SM4 recurrence:

$$K_i = K_{i+4} \oplus T'(K_{i+1} \oplus K_{i+2} \oplus K_{i+3} \oplus CK_i). \tag{7.16}$$

Iterating backward yields K_0, K_1, K_2, K_3. Undo the fixed initial constants FK_i via $MK_i = K_i \oplus FK_i$ and concatenate the four words to recover the 128-bit master key.

7.8 Experimental implications

A software simulation can inject an exact XOR mask at a chosen intermediate value. On an STM32 target, a voltage or clock glitch may instead alter multiple bits, skip an instruction, affect control flow, or produce no useful fault. A practical evaluation should therefore log correct and faulty ciphertexts, classify observed fault patterns, verify whether the assumed location and single-bit model are plausible, and report recovery success separately from simulation-only results.

Chapter 8

AES versus SM4: Key Lessons

Property	AES-128	SM4
Block size	128 bits	128 bits
Key size	128 bits	128 bits
Rounds	10	32
State	4×4 byte matrix	Four 32-bit words
Nonlinearity	AES S-box	SM4 S-box
Linear diffusion	MixColumns and ShiftRows	Rotations and XOR in L
Last-round target	K_{10}, 128 bits	rk_{31}, 32 bits
Master-key recovery	Reverse AES expansion	Reverse SM4 expansion from four consecutive round keys

Summary

For AES Giraud, inject a one-bit fault before final-round SubBytes, test inverse-S-box differences for Hamming weight one, and intersect key-byte candidates. The corresponding SM4 adaptation first inverts the linear map L to isolate the S-box differential.

For AES Piret–Quisquater, compare correct and faulty ciphertexts, invert the last S-box under candidate K_{10} bytes, match round-9 MixColumns differential patterns, intersect candidates, and reverse the AES key schedule.

For the illustrated SM4 model, compare correct and faulty ciphertexts, invert the linear map L, test S-box input candidates using the known fault differences, derive rk_{31}, and obtain further subkeys with additional appropriate experiments. Once four consecutive round keys are known, the SM4 key schedule can be reversed exactly.

Chapter 9

Python Implementation and Reproducible Terminal Results

This chapter connects the preceding mathematical derivations to the actual Python project provided with this report. The listings below are *excerpts of the project source files*, not pseudocode. The console blocks were captured by executing the supplied scripts on 24 October 2026. These are software simulations with a controllable fault observer, not physical STM32 measurements.

9.1 Repository structure and execution

The principal modules are `aes.py` (AES operations), `sm4.py` (SM4 operations), `piret.py` (DFA and round peeling), `giraud.py` (single-bit candidate filtering), and `run.py` (combined demonstration). The repository stores these modules in `src/`; `src/gui.py` provides the interactive interface. The commands below are run from the repository root.

```
python src/giraud.py --algorithm aes --known-bit
python src/giraud.py --algorithm sm4 --known-bit
python src/piret.py --algorithm sm4
```

The project also supports an unknown-bit model by omitting `-known-bit`. The `-known-bit` option is used for the reproducible outputs reproduced here.

9.2 AES-128 implementation: S-box, diffusion, and key expansion

The following code implements AES finite-field multiplication and column mixing. This is the operation responsible for the characteristic four-byte differential exploited by Piret–Quisquater.

AES finite-field arithmetic, from aes.py

```
def xtime(a):
    """Multiply a byte by x (02) in GF(2^8), reducing by polynomial 0x11B."""
    result = a << 1

    # If the x^8 term overflows, reduction contributes 0x1B to the low byte.
    if a & 0x80:
        result ^= 0x1B

    return result & 0xFF


def gf_mul(a, b):
    """Multiply bytes in the AES field; field addition is XOR, not integer +."""
    result = 0

    for _ in range(8):
```

```
        if b & 1:
            result ^= a

        # Shift-and-add multiplication, using xtime for polynomial reduction.
        a = xtime(a)
        b >>= 1

    return result


# ============================================================================
# 5. ROUND TRANSFORMATIONS: ALL MATRIX OPERATIONS ARE IN PLACE
# ============================================================================
```

AES MixColumns, from aes.py

```
def mix_column(column):
    """Return M * column in GF(2^8), without modifying the four input bytes.

    Copying a column before mixing matters: every output byte must use the same
    original input, never a value already overwritten earlier in the operation.
    """
    result = []
    for coefficients in MIX_MATRIX:
        value = 0
        for coefficient, byte in zip(coefficients, column):
            value ^= gf_mul(coefficient, byte)
        result.append(value)
    return result


def mix_columns(state):
    """Mix each column in place; the normal algorithm never prints output."""
    for column in range(4):
        mixed = mix_column([state[row][column] for row in range(4)])
        for row in range(4):
            state[row][column] = mixed[row]
```

AES round-key expansion, from aes.py

```
def key_expansion(key):
    """Expand the 16-byte master key into eleven independent round-key matrices.

    Each new word XORs the word four positions back with the previous word.
    Every fourth word first applies RotWord, SubWord and the round constant.
    """
    validate_block(key, "AES-128 key")
    # Four initial words become 44 words: K0 plus ten encryption round keys.
    words = [list(key[4 * i:4 * i + 4]) for i in range(4)]

    for i in range(4, 4 * (N_ROUNDS + 1)):
        temp = words[i - 1].copy()

        if i % 4 == 0:
            # Rcon changes only the first byte of the transformed key word.
            temp = sub_word(rot_word(temp))
            temp[0] ^= RCON[i // 4]

        new_word = [words[i - 4][j] ^ temp[j] for j in range(4)]

        words.append(new_word)

    round_keys = []

    for round_number in range(N_ROUNDS + 1):
        # Convert four consecutive words into the same layout as the data state.
        key_bytes = []

        for i in range(4):
            key_bytes.extend(words[round_number * 4 + i])
```

```
        round_keys.append(bytes_to_state(key_bytes))

    return round_keys


# ============================================================================
# 7. NORMAL ENCRYPTION / DECRYPTION AND OPTIONAL EXTERNAL OBSERVATION
# ============================================================================
```

9.3 SM4 implementation: rotations, S-boxes, and 32 rounds

The SM4 code keeps the encryption diffusion L distinct from the key-schedule diffusion L'. The observer hook allows the test harness to modify an intermediate value at a chosen round.

SM4 rotations, diffusion, and substitution, from sm4.py

```
def rotl(val: int, count: int) -> int:
    """Rotate a 32-bit unsigned word left; overflow wraps into the low bits."""
    return ((val << count) & WORD_MASK) | (val >> (32 - count))


def linear(b: int) -> int:
    """Data diffusion L: XOR the word with rotations by 2, 10, 18 and 24.

    L is XOR-linear: L(a XOR b) = L(a) XOR L(b). The SM4 DFA uses this
    property to invert an observed output difference before testing the S-box.
    """
    return b ^ rotl(b, 2) ^ rotl(b, 10) ^ rotl(b, 18) ^ rotl(b, 24)


def linear_key(b: int) -> int:
    """Key-schedule diffusion L-prime: rotations 13 and 23, not data-map L."""
    return b ^ rotl(b, 13) ^ rotl(b, 23)


def sbox(a: int) -> int:
    """Apply tau: substitute all four bytes, preserving big-endian word order."""
    input_bytes = struct.pack('>I', a)
    substituted_bytes = bytes(SBOX[byte] for byte in input_bytes)
    return struct.unpack('>I', substituted_bytes)[0]


# ============================================================================
# 4. SINGLE-ROUND RECURRENCES
# ============================================================================
```

SM4 round function and round-key generation, from sm4.py

```
def round(x: list[int], rk: int, observer=None, round_index=0):
    """Return the next rolling window without modifying the supplied words.

    A = X[i+1] XOR X[i+2] XOR X[i+3] XOR rk[i]
    X[i+4] = X[i] XOR L(tau(A))
    """
    sbox_input = observe(observer, round_index, "Sbox input", x[1] ^ x[2] ^ x[3] ^ rk)
    substituted = observe(observer, round_index, "Sbox output", sbox(sbox_input))
    diffused = observe(observer, round_index, "Linear output", linear(substituted))
    new_word = observe(observer, round_index, "New word", x[0] ^ diffused)
    return x[1], x[2], x[3], new_word


def generate_round_key(k: list[int], round: int) -> int:
    """Compute K[i+4] from four consecutive schedule words and CK[i].

    The parameter name round is retained for callers using keyword arguments.
    Encryption and expansion share tau, but expansion must use linear_key.
    """
    return k[0] ^ linear_key(sbox(k[1] ^ k[2] ^ k[3] ^ CK[round]))
```

```
# ============================================================================
# 5. KEY EXPANSION: FOUR MASTER WORDS TO 32 ROUND KEYS
# ============================================================================


def key_expansion(key: bytes) -> list[int]:
    """Return rk0..rk31; each is one new word from the key recurrence."""
    if len(key) != BLOCK_SIZE:
        raise ValueError("SM4 key must contain exactly 16 bytes.")
    master_words = struct.unpack(">4I", key)
    schedule_words = [master_words[i] ^ FK[i] for i in range(4)]

    for i in range(N_ROUNDS):
        # Keep previous words for readability; the recurrence consumes a window.
        next_key = generate_round_key(schedule_words[i:i+4], i)
        schedule_words.append(next_key)

    # The initial four whitened words are not encryption round keys.
    return schedule_words[4:]


# ============================================================================
# 6. SHARED BLOCK PROCESSING AND PUBLIC ENCRYPT/DECRYPT WRAPPERS
# ============================================================================
```

9.4 AES Giraud: filtering last-round key candidates

For each possible final-round key byte, the implementation undoes the final AddRoundKey and S-box, and tests whether the recovered input difference is a single bit. The solver does not receive the master key.

AES Giraud candidate test, from giraud.py

```
def aes_candidates(correct, faulty, known_mask=None):
    """Return the affected output position and its compatible K10 byte values.

    For each guess k:
      delta = InvSbox[C[j] XOR k] XOR InvSbox[F[j] XOR k]
    Keep k when delta has exactly one set bit, or equals the known bit mask.
    """
    aes.validate_block(correct, "Correct ciphertext")
    aes.validate_block(faulty, "Faulty ciphertext")
    if known_mask is not None:
        validate_mask(known_mask)

    # Without final-round MixColumns, only one ciphertext byte should change.
    changed = [index for index in range(16) if correct[index] != faulty[index]]
    if len(changed) != 1:
        raise ValueError("AES Giraud requires exactly one changed ciphertext byte.")
    position = changed[0]
    allowed = SINGLE_BITS if known_mask is None else (known_mask,)

    candidates = set()
    for guess in range(256):
        clean_input = AES_INVERSE_SBOX[correct[position] ^ guess]
        faulty_input = AES_INVERSE_SBOX[faulty[position] ^ guess]
        difference = clean_input ^ faulty_input
        if difference in allowed:
            candidates.add(guess)
    return position, candidates
```

9.4.1 Observed AES terminal output

The following excerpt is from an actual execution of `python src/giraud.py -algorithm aes -known-bit`. It shows the filtering process and final verification.

Recorded AES Giraud execution excerpt

```
============================================================================
```

```
GIRAUD / AES-128
Correct ciphertext: 69C4E0D86A7B0430D8CDB78070B4C55A
Bit known to solver: True
AES: require HW(InvSbox[C XOR k] XOR InvSbox[F XOR k]) = 1.

Key byte 0, observation 1, simulator mask 01
Correct: 69 C4 E0 D8 6A 7B 04 30 D8 CD B7 80 70 B4 C5 5A
Faulty:  76 C4 E0 D8 6A 7B 04 30 D8 CD B7 80 70 B4 C5 5A
XOR:     1F 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
Candidate count: 256 -> 4
Survivors: 0A 0C 13 15

Key byte 0, observation 2, simulator mask 02
Correct: 69 C4 E0 D8 6A 7B 04 30 D8 CD B7 80 70 B4 C5 5A
Faulty:  1B C4 E0 D8 6A 7B 04 30 D8 CD B7 80 70 B4 C5 5A
XOR:     72 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
Candidate count: 4 -> 1
Survivors: 13

Key byte 13, observation 1, simulator mask 01
Correct: 69 C4 E0 D8 6A 7B 04 30 D8 CD B7 80 70 B4 C5 5A
Faulty:  69 C4 E0 D8 6A 7B 04 30 D8 CD B7 80 70 83 C5 5A
XOR:     00 00 00 00 00 00 00 00 00 00 00 00 00 37 00 00
Candidate count: 256 -> 2
Survivors: 1C 2B

Key byte 13, observation 2, simulator mask 02
Correct: 69 C4 E0 D8 6A 7B 04 30 D8 CD B7 80 70 B4 C5 5A
Faulty:  69 C4 E0 D8 6A 7B 04 30 D8 CD B7 80 70 7B C5 5A
XOR:     00 00 00 00 00 00 00 00 00 00 00 00 00 CF 00 00

... [intermediate observations omitted from this printed excerpt]

XOR:     00 00 00 9A 00 00 00 00 00 00 00 00 00 00 00 00
Candidate count: 2 -> 1
Survivors: 7F

Recovered K10: 13111D7FE3944A17F307A78B4D2B30C5
Recovered master key: 000102030405060708090A0B0C0D0E0F
Verification: PASS / 32 faulty outputs
```

9.5 SM4 Giraud-inspired filtering

SM4 requires an extra step: invert the linear transformation L to isolate the S-box output difference. The following code tests candidate S-box inputs, then translates them into candidates for a byte of the last-round subkey using the three unchanged state words.

SM4 S-box differential candidate filter, from giraud.py

```
def sm4_candidates(correct, faulty, known_mask=None):
    """Return affected byte, round-key guesses, input guesses and DeltaB.

    Inputs may be actual final ciphertexts or peeled views at an earlier round.

    1. Observe DeltaX35 from the first output word.
    2. Undo linear diffusion: DeltaB = L-inverse(DeltaX35).
    3. Keep a if S(a) XOR S(a XOR e) matches the observed byte difference.
    4. Convert a into a key guess using the three unchanged output words.
    """
    aes.validate_block(correct, "Correct ciphertext")
    aes.validate_block(faulty, "Faulty ciphertext")
    if known_mask is not None:
        validate_mask(known_mask)

    # SM4 outputs X35 || X34 || X33 || X32; only X35 may change in this model.
    if correct[4:] != faulty[4:]:
        raise ValueError("SM4 last-round model requires three output words unchanged.")
    delta_x35 = int.from_bytes(correct[:4], "big") ^ int.from_bytes(faulty[:4], "big")
    delta_b = inverse_linear(delta_x35)
    differences = delta_b.to_bytes(4, "big")
    changed = [index for index in range(4) if differences[index] != 0]
```

```
    if len(changed) != 1:
        raise ValueError("SM4 inverse-L difference must affect one S-box byte.")
    position = changed[0]
    allowed = SINGLE_BITS if known_mask is None else (known_mask,)

    # The input value a is unknown. Test every value against each permitted bit.
    inputs = set()
    for guess in range(256):
        for error in allowed:
            difference = sm4.SBOX[guess] ^ sm4.SBOX[guess ^ error]
            if difference == differences[position]:
                inputs.add(guess)
                break

    # A = X32 XOR X33 XOR X34 XOR rk31; all three data words are observable.
    x34 = int.from_bytes(correct[4:8], "big")
    x33 = int.from_bytes(correct[8:12], "big")
    x32 = int.from_bytes(correct[12:16], "big")
    known_byte = ((x32 ^ x33 ^ x34) >> (24 - 8 * position)) & 0xFF
    candidates = {guess ^ known_byte for guess in inputs}
    return position, candidates, inputs, delta_b
```

9.5.1 Observed SM4 terminal output

The first fault produces a 32-bit ciphertext difference; the inverse-linear operation reduces it to a one-byte S-box difference. A second independent observation reduces the key-byte candidates from two to one.

Recorded SM4 Giraud-inspired execution excerpt

```
================================================================================
GIRAUD-INSPIRED / SM4
Correct ciphertext: 681EDF34D206965E86B3E94F536E4246
Bit known to solver: True
SM4: invert L, then test S(a) XOR S(a XOR e) for one-bit errors.

Round 31, key byte 0, observation 1
Correct: 68 1E DF 34 D2 06 96 5E 86 B3 E9 4F 53 6E 42 46
Faulty:  1B 50 E2 09 D2 06 96 5E 86 B3 E9 4F 53 6E 42 46
XOR:     73 4E 3D 3D 00 00 00 00 00 00 00 00 00 00 00 00
Undo recovered later rounds: {}
Correct: 68 1E DF 34 D2 06 96 5E 86 B3 E9 4F 53 6E 42 46
Faulty:  1B 50 E2 09 D2 06 96 5E 86 B3 E9 4F 53 6E 42 46
XOR:     73 4E 3D 3D 00 00 00 00 00 00 00 00 00 00 00 00
Candidates: 256 -> 2

Round 31, key byte 0, observation 2
Correct: 68 1E DF 34 D2 06 96 5E 86 B3 E9 4F 53 6E 42 46
Faulty:  ED 9D D9 32 D2 06 96 5E 86 B3 E9 4F 53 6E 42 46
XOR:     85 83 06 06 00 00 00 00 00 00 00 00 00 00 00 00
Undo recovered later rounds: {}
Correct: 68 1E DF 34 D2 06 96 5E 86 B3 E9 4F 53 6E 42 46
Faulty:  ED 9D D9 32 D2 06 96 5E 86 B3 E9 4F 53 6E 42 46
XOR:     85 83 06 06 00 00 00 00 00 00 00 00 00 00 00 00
Candidates: 2 -> 1

Round 31, key byte 1, observation 1
Correct: 68 1E DF 34 D2 06 96 5E 86 B3 E9 4F 53 6E 42 46
Faulty:  AD AB AF F1 D2 06 96 5E 86 B3 E9 4F 53 6E 42 46
XOR:     C5 B5 70 C5 00 00 00 00 00 00 00 00 00 00 00 00
Undo recovered later rounds: {}

... [intermediate observations omitted from this printed excerpt]

Candidates: 2 -> 1
Recovered rk31: 9124A012
Recovered rk30: 01CF72E5
Recovered rk29: 62293496
Recovered rk28: 428D3654
Recovered master key: 0123456789ABCDEFFEDCBA9876543210
Verification: PASS / 32 faulty outputs / no reference subkeys
```

9.6 Recovering SM4 rounds 31 through 28

The source project implements an additional *multi-round simulated attack*, beyond the last-round example described earlier in this report. Once rk_{31} is recovered, its value is used to undo the last round in both correct and faulty ciphertexts. A separate controlled fault at round 30 can then be analyzed as a last-round differential on the reconstructed state. The same process is repeated for rounds 29 and 28.

Crucially, the algorithm does *not* supply reference round keys to the recovery function. It uses previously recovered keys to peel later rounds. However, its fault oracle must be capable of injecting faults at each of the four targeted rounds, a strong capability not established by these software tests on physical hardware.

Undoing already recovered SM4 rounds, from piret.py

```
def peel_sm4(ciphertext, target_round, recovered_keys):
    """Undo only rounds whose subkeys have ALREADY been recovered by DFA.

    Return a reversed four-word output at the end of target_round. This lets
    the same last-round equations analyze rounds 30, 29 and 28 in succession.
    No reference round keys or secret intermediate values are accepted.
    """
    aes.validate_block(ciphertext, "SM4 ciphertext")
    if not 28 <= target_round <= 31:
        raise ValueError("Full SM4 recovery targets rounds 28..31.")
    words = list(struct.unpack(">4I", ciphertext))[::-1]
    for number in range(31, target_round, -1):
        if number not in recovered_keys:
            raise ValueError(f"Cannot undo round {number} before recovering its subkey.")
        previous = words[3] ^ sm4.linear(sm4.sbox(words[0] ^ words[1] ^ words[2] ^
        ↪  recovered_keys[number]))
        words = [previous, words[0], words[1], words[2]]
    return struct.pack(">4I", *words[::-1])
```

Sequential recovery of four SM4 subkeys, from piret.py

```
def recover_sm4_master_key(correct, fault_oracle, masks=(1, 2, 3, 4, 5, 6, 7, 8), start_byte=0,
↪  candidate_filter=None, observer=None):
    """Recover rk31 -> rk28 and reverse the schedule, using outputs only.

    fault_oracle(round, byte, mask) returns a faulty ciphertext. It represents
    a target capable of a controlled fault in EACH of the final four rounds.
    The solver receives no master key, schedule, or reference subkeys.
    The default filter uses known masks; Giraud supplies its single-bit filter.
    """
    aes.validate_block(correct, "Correct SM4 ciphertext")
    masks = tuple(masks)
    if not masks or len(set(masks)) != len(masks) or any(not 1 <= mask <= 255 for mask in masks) or not
    ↪  0 <= start_byte < 4:
        raise ValueError("Use distinct nonzero byte masks and start_byte 0..3.")
    keys, history, counts, used = {}, [], {}, 0
    for number in range(31, 27, -1):
        clean_view = peel_sm4(correct, number, keys)
        subkey = bytearray(4)
        for byte in [(start_byte + offset) % 4 for offset in range(4)]:
            survivors = None
            for observation, mask in enumerate(masks, 1):
                faulty = bytes(fault_oracle(number, byte, mask))
                faulty_view = peel_sm4(faulty, number, keys)
                if candidate_filter is None:
                    fresh, inputs, delta = sm4_byte_candidates(clean_view, faulty_view, byte, mask)
                else:
                    position, fresh, inputs, delta = candidate_filter(clean_view, faulty_view, mask)
                    if position != byte:
                        raise ValueError("Fault observation does not match the requested byte.")
                before = 256 if survivors is None else len(survivors)
                survivors = set(fresh) if survivors is None else survivors & fresh
                if not survivors:
                    raise ValueError("SM4 candidate intersection is empty; fault model is
                    ↪  incompatible.")
```

```
                used += 1
                counts[f"{number}:{byte}"] = len(survivors)
                event = dict(round=number, byte=byte, observation=observation, mask=mask,
                ↪  before=before, pair_candidates=len(fresh), remaining=len(survivors),
                ↪  survivors=sorted(survivors), inputs=sorted(inputs), delta_b=delta,
                ↪  correct=correct.hex(), faulty=faulty.hex(), clean_view=clean_view.hex(),
                ↪  faulty_view=faulty_view.hex(), peeled_with={str(i): f"{value:08X}" for i, value in
                ↪  keys.items()})
                history.append(event)
                if observer is not None:
                    observer(dict(event))
                if len(survivors) == 1:
                    subkey[byte] = next(iter(survivors))
                    break
            else:
                return dict(master_key=None, round_keys=keys, history=history, candidate_counts=counts,
                ↪  used=used, complete=False, blocked_round=number)
        keys[number] = int.from_bytes(subkey, "big")
    master = reverse_sm4([keys[number] for number in range(28, 32)])
    return dict(master_key=master, round_keys=keys, history=history, candidate_counts=counts,
    ↪  used=used, complete=True)
```

9.7 Reversing the SM4 key schedule

After recovering four consecutive 32-bit subkeys, the code reverses the key schedule and removes the initial FK constants.

Reverse SM4 key expansion, from piret.py

```
def reverse_sm4(final_keys, observer=None):
    """Four consecutive keys rk28..rk31 are REQUIRED, not merely rk31."""
    if len(final_keys) != 4:
        raise ValueError("SM4 reversal requires four consecutive final subkeys.")
    words = [None] * 32 + list(final_keys)
    for index in range(31, -1, -1):
        value = words[index+1] ^ words[index+2] ^ words[index+3] ^ sm4.CK[index]
        words[index] = words[index+4] ^ sm4.linear_key(sm4.sbox(value))
        if observer:
            observer(index, words[index])
    return struct.pack(">4I", *(words[i] ^ sm4.FK[i] for i in range(4)))
```

9.7.1 Independent terminal validation

The controlled-byte model also runs independently in `piret.py`. The following excerpt shows the real output of the supplied code:

Recorded SM4 controlled-byte DFA execution excerpt

```
================================================================================
SM4: NORMAL ENCRYPTION
Round 00: A=F002C39E B=18E992B1 L(B)=26D99622 new=27FAD345
Round 01: A=EE1470AC B=C62AD40D L(B)=2820815D new=A18B4CB2
Round 02: A=AA4F1C7D B=23A849C4 L(B)=EF1D58B2 new=11C1E22A
Round 03: A=EC195DAA B=C5445623 L(B)=BA47D0FE new=CC13E2EE
Round 04: A=4A2A2C82 B=3C0BED8A L(B)=DF868890 new=F87C5BD5
Round 05: A=52C45770 B=81BB8BD4 L(B)=92A94BE5 new=33220757
Round 06: A=B1F637DF B=DB4D95B0 L(B)=663520BD new=77F4C297
Round 07: A=98DCAF44 B=ADB86FF3 L(B)=B6851005 new=7A96F2EB
Round 08: A=9B600757 B=301EB78B L(B)=DFA69BAA new=27DAC07F
Round 09: A=9DE0BDBE B=8C896C5B L(B)=71FF084E new=42DD0F19
Round 10: A=DC966E60 B=B81A781E L(B)=CF511895 new=B8A5DA02
Round 11: A=A3474E33 B=2EBA4FA9 L(B)=EAE7D511 new=907127FA
Round 12: A=0381926D B=FEBF5D21 L(B)=AC4FEBFC new=8B952B83
Round 13: A=939943CC B=A493FC1F L(B)=96F67340 new=D42B7C59
Round 14: A=8B75648F B=CED363A1 L(B)=97598233 new=2FFC5831
Round 15: A=60069A4A B=1E3D323C L(B)=66EF4F72 new=F69E6888
Round 16: A=DC69F8C8 B=B8227911 L(B)=24B11947 new=AF2432C4
Round 17: A=05F35DDE B=E1EC56B4 L(B)=3935B407 new=ED1EC85E
Round 18: A=7823DB74 B=4CF4129F L(B)=7A5FE213 new=55A3BA22
```

```
Round 19: A=85BD0481 B=C76CCCBF L(B)=E4D57022 new=124B18AA
Round 20: A=42680EC9 B=A7252CD9 L(B)=C5C3409B new=6AE7725F
Round 21: A=B5C5D18D B=FDDDC161 L(B)=19D569A7 new=F4CBA1F9
Round 22: A=4B725B6C B=19464B01 L(B)=486E4032 new=1DCDFA10
Round 23: A=1A00D498 B=13D6A5AD L(B)=3DBD1EA9 new=2FF60603
Round 24: A=716B85E6 B=003BC777 L(B)=85153D83 new=EFF24FDC
Round 25: A=C0E8A67F B=8D65CA9E L(B)=9B2FCA8C new=6FE46B75
Round 26: A=A1C2A841 B=F6AFC007 L(B)=94F9AABD new=893450AD
Round 27: A=F85A7885 B=790F4CC7 L(B)=5465894F new=7B938F4C
Round 28: A=DFCE82C0 B=B05A8A8D L(B)=BC9C0D9A new=536E4246

... [intermediate observations omitted from this printed excerpt]

Survivors: 54
DFA RECOVERED rk31 = 9124A012 / PASS
DFA RECOVERED rk30 = 01CF72E5 / PASS
DFA RECOVERED rk29 = 62293496 / PASS
DFA RECOVERED rk28 = 428D3654 / PASS
SM4 MASTER KEY: 0123456789ABCDEFFEDCBA9876543210 / PASS
32 faulty outputs. All four subkeys came from DFA, not references.
```

9.8 Interpretation of the results

The terminal output demonstrates successful *software-model* recovery of the AES-128 master key and the SM4 master key. In the SM4 Giraud-inspired known-bit demonstration, the program reports 32 faulty outputs, the four recovered round keys, and a final successful verification. The corresponding values are:

$$rk_{31} = \texttt{9124A012}, \qquad rk_{30} = \texttt{01CF72E5},$$
$$rk_{29} = \texttt{62293496}, \qquad rk_{28} = \texttt{428D3654},$$
$$MK = \texttt{0123456789ABCDEFFEDCBA9876543210}.$$

This extends the earlier pedagogical last-round discussion: the supplied project *does* demonstrate four-subkey recovery under its simulated, selectable-round fault oracle. It does *not* establish that the same fault precision or success rate is achievable through ChipWhisperer glitches on the STM32 target.

Chapter 10

Complete Code Walkthrough: Every Implemented Attack

This chapter documents all attack paths currently implemented for the two project ciphers, AES-128 and SM4. It connects the fault location, observable data, candidate equations, Python functions and recovery of the original 128-bit key. The excerpts are taken from the current repository source. The earlier numerical last-round examples remain useful illustrations; the selectable-round SM4 implementation extends their capabilities under an additional fault-control assumption.

10.1 Which attacks are implemented?

Cipher and model	Fault location	Solver knowledge	Full-key route
AES Piret-style	Before round-9 MixColumns; also a round-8 variant	Correct/faulty ciphertexts and output group; unknown error magnitude and row	Recover all four K_{10} quartets, then reverse the AES-128 schedule
AES Giraud	Before round-10 SubBytes	Correct/faulty ciphertexts; one bit, with optional exact mask	Recover all sixteen K_{10} bytes, then reverse the AES-128 schedule
SM4 controlled-byte DFA	S-box input in rounds 31, 30, 29, 28	Correct/faulty outputs and specified byte and XOR mask	Recover rk_{31} down to rk_{28}, then reverse the SM4 schedule
SM4 Giraud-inspired	Same four SM4 rounds; one-bit masks	Correct/faulty outputs; one bit, with optional exact mask	Apply bit filtering, recover the four final subkeys and reverse the schedule

Note. The SM4 byte attack lives in `piret.py`, but it is a controlled SM4 differential model, not the AES Piret equations applied unchanged. The SM4 single-bit model is likewise an adaptation of Giraud's filtering idea. The project does not implement every published fault attack, AES-192, or AES-256.

AES round numbers here are 1–10 after the initial key XOR; SM4 indices are zero-based, 0–31. Each model attacks encryption. Decryption in the GUI checks the normal cipher and is not a separate fault-recovery attack.

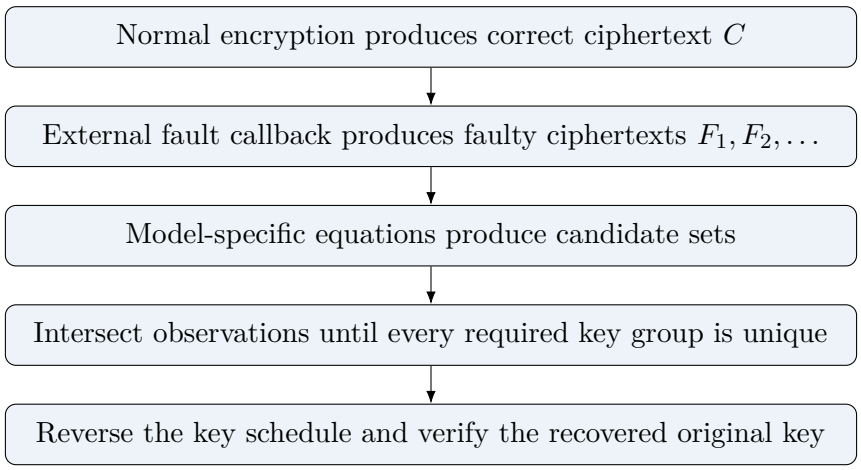
10.2 Normal cipher, simulated target, solver, and viewer

1. **Normal cipher:** `aes.aes_encrypt` and `sm4.sm4_encrypt_block` execute the actual algorithm. Their optional observers expose transformation events. Without an attack callback, these functions make no fault-dependent modification.

2. **Target adapter:** the attack module attaches a callback that changes a chosen intermediate value. The adapter needs the simulation key to operate the target. This is analogous to a physical target possessing its own key.

3. **Candidate solver:** it receives the observations allowed by its model, not the target's key or expanded schedule. Traces of hidden states are teaching data, not candidate-filter inputs.

4. **Presentation and checks:** `gui.py` builds timelines, highlights differences in red, and checks the recovered key by re-encryption. A separate comparison with the known simulation key is a test of the experiment, not a source of recovered key bytes.



10.3 AES Piret-style attack: code from injection to the master key

10.3.1 Inject into the real AES implementation

`encrypt_with_trace` in `piret.py` installs an observer on `aes.aes_encrypt`. The callback waits for `Before MixColumns` in the selected round and performs `state[row][column] ^= mask`. It then saves immutable byte snapshots. A zero mask gives a clean trace; nonzero masks represent controlled software faults. The solver does not receive this mask.

External Piret fault injection into the real AES engine

```
def encrypt_with_trace(plaintext, key, row=0, column=0, mask=0, fault_round=8):
    """Run aes.py itself; this external observer injects and records the fault.

    Round 8 spreads to four key groups. Round 9 reaches one key quartet.
    No second AES implementation is used by the attacker.
    """
    if not (0 <= row < 4 and 0 <= column < 4 and 0 <= mask <= 255 and fault_round in (8, 9)):
        raise ValueError("Fault: row/column 0..3, mask 0..255, round 8 or 9.")
    trace = []

    def observer(number, operation, state):
        if number == fault_round and operation == "Before MixColumns":
            state[row][column] ^= mask
            trace.append((number, "Fault point", aes.state_to_bytes(state)))
```

```
            if number > 0 and operation.startswith("After "):
                trace.append((number, operation[6:], aes.state_to_bytes(state)))

        ciphertext = aes.aes_encrypt(plaintext, key, observer=observer)
        return ciphertext, trace
```

10.3.2 Convert one output pair into four-byte key candidates

For column c, the output indices are $p_r = 4((c - r) \bmod 4) + r$, $r = 0, 1, 2, 3$. Under a guess k_r for a final-key byte,

$$d_r(k_r) = S^{-1}(C_{p_r} \oplus k_r) \oplus S^{-1}(F_{p_r} \oplus k_r).$$

The code first groups the 256 byte guesses by their resulting difference. It then enumerates the *shared* unknown error row q and error value e, requiring all four equations $d_r(k_r) = M_{r,q}e$ at once. The Cartesian product combines only compatible byte lists; it does not independently choose arbitrary surviving bytes. This preserves the quartet correlation without enumerating all 2^{32} quartets first.

Piret candidate quartets with one shared row and error

```
def key_candidates(correct, faulty, column):
    """Derive four-byte K10 candidates from ciphertexts only.

    For each guessed byte k, undo final AddRoundKey and SubBytes:
      d = InvSbox[C[p] XOR k] XOR InvSbox[Cfault[p] XOR k].
    All four differences must fit ONE matrix column times ONE unknown error.
    """
    if len(correct) != 16 or len(faulty) != 16:
        raise ValueError("Ciphertexts must each contain exactly 16 bytes.")
    # Each table maps a possible input difference d to compatible byte guesses.
    # Four small lookups avoid brute-force enumeration of all 2^32 quartets.
    tables = []
    for position in output_positions(column):
        table = [[] for _ in range(256)]
        for guess in range(256):
            delta = INV_SBOX[correct[position] ^ guess] ^ INV_SBOX[faulty[position] ^ guess]
            table[delta].append(guess)
        tables.append(table)
    found = set()
    for unknown_row in range(4):
        for unknown_error in range(1, 256):
            # A single shared row/error must satisfy ALL four equations.
            # Independent byte filtering alone would lose this constraint.
            choices = [tables[r][gf_multiply(MIX_MATRIX[r][unknown_row], unknown_error)] for r in
            ↪  range(4)]
            found.update(product(*choices))
    return found
```

10.3.3 Round 9 versus the supported round-8 variant

The GUI's `pdf_aes_lesson` and the terminal's `run.aes_demo` use round 9. Each observation affects one output quartet. These functions intersect `key_candidates` sets separately for all four columns and place the unique tuples back into their original K_{10} byte positions.

The direct `piret.py` AES command uses round 8. The additional round spreads the original byte disturbance to all four column groups; each final-round group still satisfies a round-9 one-byte-input differential equation with its own effective error. That effective error is not necessarily the original injected mask. `recover_last_key` therefore processes all four groups from each faulty output. Its explicit requirement that all sixteen ciphertext bytes change belongs to this round-8 path; it must not be imposed on round-9 observations.

Round-8 AES recovery: intersect all four output groups

```
def recover_last_key(correct, faulty_outputs, observer=None):
    """Attacker API: no master key, plaintext, fault mask or internal states.
```

```
    The optional observer is for logging only; it cannot modify candidate sets.
    A None result means ambiguity, never an arbitrary candidate selection.
    """
    if len(correct) != 16:
        raise ValueError("Correct ciphertext must contain 16 bytes.")
    groups, used = [None] * 4, 0
    for used, faulty in enumerate(faulty_outputs, 1):
        if len(faulty) != 16 or any(a == b for a, b in zip(correct, faulty)):
            raise ValueError("Expected 16 changed output bytes for the round-8 byte-fault model.")
        for column in range(4):
            fresh = key_candidates(correct, faulty, column)
            before = None if groups[column] is None else len(groups[column])
            # An intersection can retain/remove candidates, never introduce new ones.
            groups[column] = fresh if groups[column] is None else groups[column] & fresh
            if not groups[column]:
                raise ValueError("Empty candidate intersection: incompatible ciphertexts or fault
                ↪  model.")
            if observer:
                observer(used, column, before, len(fresh), tuple(sorted(groups[column])))
        if all(len(group) == 1 for group in groups):
            # Group tuples are in pre-ShiftRows row order. Restore true K10 order.
            last_key = bytearray(16)
            for column in range(4):
                for position, byte in zip(output_positions(column), next(iter(groups[column]))):
                    last_key[position] = byte
            return bytes(last_key), used
    return None, used
```

10.3.4 Recover the original AES key

Once K_{10} is unique, `reverse_key_schedule` initializes $w_{40}, \ldots, w_{43}$ and walks downward. It computes each earlier word with XOR and the forward key-schedule transformation. It never reads the original key. The forward S-box is correct here because its input schedule word is already known.

Reverse AES-128 expansion to recover the original key

```
def reverse_key_schedule(last_key, observer=None):
    """AES-128 recurrence reversed: w[i-4] = w[i] XOR transform(w[i-1]).

    Descending order guarantees that both right-hand words are already known.
    Reversal still uses the FORWARD S-box: its input word is known, so there
    is no reason to invert SubWord. This function never reads the master key.
    """
    if len(last_key) != 16:
        raise ValueError("K10 must contain 16 bytes.")
    words = [None] * 44
    # K10 occupies the final four schedule words, w40 through w43.
    words[40:] = [list(last_key[index:index+4]) for index in range(0, 16, 4)]
    for index in range(43, 3, -1):
        transformed = words[index - 1][:]
        if index % 4 == 0:
            # Same RotWord/SubWord/Rcon transformation as forward expansion.
            transformed = [SBOX[byte] for byte in transformed[1:] + transformed[:1]]
            transformed[0] ^= aes.RCON[index // 4]
        words[index - 4] = [a ^ b for a, b in zip(words[index], transformed)]
        if observer:
            observer(index, bytes(words[index]), bytes(transformed), bytes(words[index - 4]))
    return bytes(sum(words[:4], []))
```

If a group still has more than one candidate, the lesson reports ambiguity instead of guessing a key. Increasing the number of observations can help, but no fixed success count is guaranteed for every target state and fault sequence.

10.4 AES Giraud attack: one-bit filtering and complete recovery

10.4.1 External fault before final SubBytes

`giraud.aes_fault_encrypt` changes one bit in the state immediately before final-round SubBytes. Byte index b maps to `state[b % 4][b // 4]`, consistent with AES column-major storage. Unlike the Piret fault, no MixColumns follows this location; one altered S-box input byte reaches one ciphertext byte after ShiftRows.

External single-bit fault before final AES SubBytes

```
def aes_fault_encrypt(plaintext, key, byte=0, mask=None):
    """Interfere with the real aes.py execution before final SubBytes.

    Only this attack callback changes the state. Without it AES is unchanged.
    Snapshots are copies for teaching; they are never inputs to the solver.
    """
    if not 0 <= byte < 16:
        raise ValueError("AES input byte must be 0..15.")
    if mask is not None:
        validate_mask(mask)
    trace = {}

    def observer(number, operation, state):
        if number == 10 and operation == "Before SubBytes":
            trace["Before fault"] = aes.state_to_bytes(state)
            if mask is not None:
                state[byte % 4][byte // 4] ^= mask
            trace["Injected bit"] = aes.state_to_bytes(state)
        if number == 10 and operation.startswith("After "):
            name = "Ciphertext" if operation == "After AddRoundKey" else operation[6:]
            trace[name] = aes.state_to_bytes(state)

    ciphertext = aes.aes_encrypt(plaintext, key, observer=observer)
    return ciphertext, trace
```

10.4.2 Why the filter works

`giraud.aes_candidates`, listed in the implementation chapter, first requires exactly one changed output byte. For each $k \in \{0, \ldots, 255\}$, it computes

$$\delta(k) = S^{-1}(C_j \oplus k) \oplus S^{-1}(F_j \oplus k).$$

In unknown-bit mode, it keeps k when $\delta(k) \in \{1, 2, 4, 8, 16, 32, 64, 128\}$. In known-bit mode, it requires the stronger condition $\delta(k) = e$ for the supplied exact mask. A one-bit difference is required at the *inverse-S-box input comparison*, not necessarily in the observed ciphertext XOR.

`giraud.masks_for` generates up to eight distinct single-bit observations per target byte. `giraud.intersect_candidates` intersects the resulting sets. The terminal `main` and GUI `giraud_lesson` repeat this for all sixteen input byte positions, use the observed output index after ShiftRows to assemble K_{10}, and call `piret.reverse_key_schedule`. Even after all eight masks, an ambiguous byte must remain reported as ambiguous. Knowing the fault location does not imply knowing its exact bit.

10.5 SM4 controlled-byte DFA: code and full-key extension

10.5.1 Inject before tau and undo output diffusion

`piret.sm4_fault_encrypt` attaches an observer to `sm4.sm4_encrypt_block`. At `Sbox input`, it applies `mask << (24 - 8*byte)` to the 32-bit value. The shift preserves big-endian byte numbering. Its diagnostic trace may contain subkeys for teaching; the oracle passed to the solver returns only the faulty ciphertext.

At target round i,

$$A_i = X_{i+1} \oplus X_{i+2} \oplus X_{i+3} \oplus rk_i, \qquad X_{i+4} = X_i \oplus L(\tau(A_i)).$$

When the round is last in the current view, only X_{i+4} changes. Since L is XOR-linear,

$$\Delta B_i = L^{-1}(X_{i+4} \oplus X'_{i+4}).$$

`inverse_rows` constructs the 32×32 binary matrix of L and inverts it by Gaussian elimination. `inverse_linear` applies the inverse through binary dot products. This is an inverse of the data diffusion map, not the key-schedule map L'.

10.5.2 Turn S-box input guesses into key-byte guesses

For the known fault byte e, enumerate a satisfying $S(a) \oplus S(a \oplus e) = \Delta B_{i,b}$. Let q be byte b of $X_{i+1} \oplus X_{i+2} \oplus X_{i+3}$, available in the other three words of the view. Then a compatible subkey byte is $a \oplus q$. The function validates the unchanged words and single-byte support of ΔB_i before filtering.

Known-byte SM4 differential filtering

```
def sm4_byte_candidates(correct, faulty, byte, mask):
    """Attacker API: outputs and the PDF's KNOWN fault byte/value only."""
    if len(correct) != 16 or len(faulty) != 16 or not 0 <= byte < 4 or not 1 <= mask <= 255:
        raise ValueError("Invalid SM4 output or known-fault descriptor.")
    c, f = struct.unpack(">4I", correct), struct.unpack(">4I", faulty)
    if c[1:] != f[1:]:
        raise ValueError("The last-round model requires three output words unchanged.")
    delta_b = inverse_linear(c[0] ^ f[0])
    shift = 24 - 8 * byte
    if not delta_b or delta_b & ~(255 << shift):
        raise ValueError("Difference does not match a single-byte S-box input fault.")
    d = (delta_b >> shift) & 255
    known = ((c[1] ^ c[2] ^ c[3]) >> shift) & 255
    inputs = {a for a in range(256) if sm4.SBOX[a] ^ sm4.SBOX[a ^ mask] == d}
    return {a ^ known for a in inputs}, inputs, delta_b
```

10.5.3 Recover four subkeys instead of supplying three reference values

A single last-round experiment recovers at most rk_{31}, which is only 32 bits. The implemented `piret.recover_sm4_master_key` extends the model:

1. Recover all four bytes of rk_{31} using faults in round 31.
2. Request new faulty outputs from round 30. Undo round 31 on *both* correct and faulty outputs using the recovered rk_{31}.
3. Apply the same last-round equations to this restored round-30 view to recover rk_{30}.
4. Repeat at round 29, then round 28, undoing only later rounds whose keys have already been recovered.
5. Pass the recovered keys in the order $[rk_{28}, rk_{29}, rk_{30}, rk_{31}]$ to `reverse_sm4`.

For a rolling window $(X_{i+1}, X_{i+2}, X_{i+3}, X_{i+4})$ and a known recovered rk_i, the previous word is

$$X_i = X_{i+4} \oplus L\big(\tau(X_{i+1} \oplus X_{i+2} \oplus X_{i+3} \oplus rk_i)\big).$$

This is the equation implemented by `peel_sm4`, already listed in the implementation chapter. The returned view uses reversed word order so that `sm4_byte_candidates` can reuse its output equations. An earlier-round fault may affect several words of the *raw* final ciphertext; the three-unchanged-word condition is checked after the appropriate peeling.

The full recovery function's inputs are a correct output, a callable `fault_oracle(round, byte, mask)`, and model configuration. It receives neither plaintext nor a key schedule. The oracle closure knows the simulation key only to run the target. If any byte remains ambiguous, the function returns `complete=False` and `master_key=None`; it never peels an unrecovered round with a reference key.

`reverse_sm4` reverses $K_{i+4} = K_i \oplus T'(K_{i+1} \oplus K_{i+2} \oplus K_{i+3} \oplus CK_i)$, using $T' = L' \circ \tau$. After obtaining $K_0, \ldots, K_3$, it XORs the fixed FK_i constants to recover the four original master-key words. Four consecutive round keys are required; knowledge of rk_{31} alone is insufficient.

10.6 SM4 Giraud-inspired attack: reuse with a different filter

`giraud.sm4_fault_encrypt` validates that the simulator mask is a single bit, then reuses the actual SM4 target adapter. `giraud.sm4_candidates`, listed earlier, computes $L^{-1}(\Delta X)$ and identifies the changed S-box byte. For each candidate input a, unknown-bit mode accepts it if

$$\exists e \in \{1, 2, 4, 8, 16, 32, 64, 128\} : \quad S(a) \oplus S(a \oplus e) = \Delta B_b.$$

Known-bit mode tests only the specified e. The surviving a values are converted to subkey candidates with the same observable-word XOR as the controlled-byte model.

The following wrapper changes only the candidate filter. It keeps the sequential round recovery, intersections, peeling, and key-schedule reversal shared with the byte-DFA engine. In particular, the parameter `simulator_mask` is deliberately not forwarded to `sm4_candidates` in unknown-bit mode.

Giraud-inspired SM4 wrapper with explicit mask knowledge

```
def recover_sm4_master_key(correct, fault_oracle, first_mask=1, maximum=8, known_bit=False,
↪  start_byte=0, observer=None):
    """Full SM4 recovery using single-bit filtering in rounds 31, 30, 29, 28."""
    import piret

    def candidate_filter(clean, faulty, simulator_mask):
        # Unknown-bit mode deliberately does not pass the oracle's mask onward.
        return sm4_candidates(clean, faulty, simulator_mask if known_bit else None)

    return piret.recover_sm4_master_key(correct, fault_oracle, masks_for(first_mask, maximum),
    ↪  start_byte, candidate_filter, observer)
```

This division explains why two attacks share a full-key recovery engine without being identical: the allowed error sets and the information supplied to the filter differ. Both require controlled faults at each of rounds 31–28 for full recovery in this implementation.

10.7 How the GUI maps to the attack code

Run `python src/gui.py` from the repository root. Select the algorithm and operation, load a standard example or enter a 16-byte hexadecimal key and block, then build the lesson.

- **Encrypt / Decrypt:** `aes_trace` or `sm4_trace` observes normal transformations. The inverse operation verifies the input block is recovered.
- **AES / Fault analysis:** `pdf_aes_lesson` uses the round-9 Piret path, processes all four columns and recovers K_0.
- **SM4 / Fault analysis:** `pdf_sm4_lesson` calls `sm4_master_lesson` with the known-byte filter.
- **Giraud attack:** `giraud_lesson` uses the AES byte filter or routes SM4 to `sm4_master_lesson` with the single-bit filter. The exact-bit checkbox selects whether the solver receives the mask.

The GUI row/byte and column fields choose the initial demonstration coordinate, not a restriction to recovering only one key byte. The maximum-observation setting is per target group or byte. A lesson snapshot stores the round, operation, explanation, 16-byte displayed state, details and optional correct reference. Normal steps compare with the previous displayed state; fault steps compare with the corresponding correct state. Red nonzero XOR bytes visualize these comparisons. SM4 scalar intermediates A, $\tau(A)$ and $L(\tau(A))$ appear in the details; the rolling state does not change until the new word is appended.

10.8 Commands and verified default examples

```
# GUI: both algorithms and both fault-analysis models
python src/gui.py
# AES round-8 Piret-style terminal lesson
python src/piret.py --algorithm aes
# AES round-9 Piret and controlled-byte SM4 DFA
python src/run.py
# Controlled-byte SM4 full master-key recovery
python src/piret.py --algorithm sm4
# Unknown-bit Giraud models (default)
python src/giraud.py --algorithm aes
python src/giraud.py --algorithm sm4
# Stronger model: exact single-bit mask given to the solver
python src/giraud.py --algorithm aes --known-bit
python src/giraud.py --algorithm sm4 --known-bit
```

On 7 November 2026, the following GUI lesson builders were executed directly without opening a window. Coordinates were zero, first mask was `01`, maximum observations was eight, and Giraud used its default unknown-bit mode. The official inputs and keys given earlier were used.

Lesson	Faulty outputs used	Recovered original key
AES round-9 Piret	8	`000102030405060708090A0B0C0D0E0F`
AES Giraud, unknown bit	35	`000102030405060708090A0B0C0D0E0F`
SM4 known-byte DFA	32	`0123456789ABCDEFFEDCBA9876543210`
SM4 Giraud-inspired, unknown bit	35	`0123456789ABCDEFFEDCBA9876543210`

All four returned `verified=True`. These counts describe these particular deterministic examples, not a bound for arbitrary keys, plaintexts, masks or physical targets. Earlier terminal excerpts use the explicitly known-bit model, which explains why their counts can differ.

10.9 Validation and limits of the implementation

The regression suite checks known-answer vectors, inverse operations, fault propagation, candidate intersections, key-schedule reversal and ambiguous cases. `tests/test_cipher_separation.py` checks that the attacks intervene in the real cipher engines. `tests/test_sm4_full_recovery.py` tests random keys with a precomputed output-only oracle and blocks `sm4.key_expansion` while the recovery solver executes. This guards against accidentally obtaining the key from a reference schedule. `tests/test_giraud.py` checks known/unknown-bit filtering and recovery for both algorithms.

```
python -m unittest discover -s tests -v
python tests/test_crypto_lab.py --gui CryptoTests.test_gui
```

The simulated observer is more precise than an uncharacterized physical glitch: it selects a round, intermediate value and bit/byte. No runtime trace or known test key is evidence that a ChipWhisperer glitch on STM32F303 satisfies that model. Physical evaluation must establish the observed fault support and location, then report useful-fault rate, ambiguity and key recovery separately. No additional hardware results are claimed by this code walkthrough.

Chapter 11

Implementation Manual: Every Function and Its Execution Order

This manual explains every named function, class method and nested callback in the six executable source modules. It complements the mathematical chapters with the actual implementation order, arguments, state changes, return values and interactions. Constants, module initialization, lambdas and main guards are explained separately. The appendix contains the complete source snapshot, including imports, tables, comments and entry points, so this document can be read without opening an external Python file.

11.1 Reading Python data and control flow

- A byte is an integer from zero to 255; immutable `bytes` are used for external blocks and stored snapshots. `list` and nested row lists hold mutable working states. Reassigning an element of an AES matrix changes that matrix; replacing a local integer requires returning the replacement.
- `^` is XOR, `&` is bitwise AND, and shifts move bits. `0xFF` keeps eight low bits and `0xFFFFFFFF` keeps thirty-two. These operators are not ordinary addition and multiplication.
- `[a:b]` copies a slice; `[::-1]` reverses order. AES row rotation moves bytes inside one row; SM4's final reversal moves four entire words. These operations must not be confused.
- `None` means no value/observer or no observation yet. A nonempty candidate set with multiple elements means ambiguity. Empty intersections are model errors, not successful recovery of zero-valued key bytes.
- An observer is a function passed to another function. A closure captures values from its containing execution. A generator yields observations only when the solver asks for them. These constructs connect the existing engines to the experiments without copying the encryption algorithms.
- Each `if __name__ == "__main__"` block runs only on direct execution. Importing a module defines its helpers and constants without automatically starting its terminal demonstration or opening a Tk window.

11.2 Module-level configuration and initialization

`aes.py` `KEY` and `PLAINTEXT` are editable demonstration strings, not implicit inputs to the block functions. `BLOCK_SIZE=16` and `N_ROUNDS=10` select AES-128. `SBOX` contains the standard

substitution table; `INV_SBOX` is constructed by reversing its mapping. `RCON` uses an unused index zero so schedule rounds index directly. `MIX_MATRIX` and `INV_MIX_MATRIX` contain the field coefficients for forward/inverse column mixing.

`sm4.py` The example strings use the official vector. `WORD_MASK` bounds words to thirty-two bits; `BLOCK_SIZE` and `N_ROUNDS` select sixteen bytes and thirty-two rounds. The SM4 `SBOX` is different from AES. Four `FK` words initialize the key schedule; thirty-two `CK` words enter the schedule recurrence. Their byte construction is documented next to the constants in the source.

`piret.py` Imports expose the real cipher engines, Cartesian products for correlated quartet candidates, a cached inverse binary matrix, random fault sequences and terminal formatting. The legacy key/plaintext/fault globals are for `relation_demo`; the main recovery path reads the cipher configuration or explicit parameters. The inverse AES S-box and mixing matrix are shared mathematical data, not a secret key.

`giraud.py` `SINGLE_BITS` enumerates the eight possible one-bit byte masks. `AES_INVERSE_SBOX` is public algorithm data. The inverse SM4 data diffusion function is imported from the common attack module. Mask knowledge is an explicit argument, not inferred from the simulator's hidden state.

`run.py` Imports the target engines and common attack mathematics. `LessonTerminal` extends `piret.Terminal` with flat byte comparisons; its inherited color and pause behavior remains presentation only.

`gui.py` Imports JSON export, queue/thread support and Tk widgets, then the cipher/attack modules. `AES_KEY`, `AES_INPUT` and `SM4_VECTOR` define examples. Color constants style the window. Its extra inverse-S-box table is module-level teaching data; candidate solving still delegates to the attack modules. The main guard creates the Tk root, constructs `CryptoLab` and enters `mainloop`.

11.3 End-to-end trace: ordinary AES and SM4

1. In AES, `aes_encrypt(P,K)` first calls `bytes_to_state(P)` and `key_expansion(K)`. The initial state byte at row one, column zero is `11`; its matching first-key byte is `01`, so the initial XOR produces `10`.

2. `_operation` applies the initial key and each subsequent transformation. At round one, `sub_bytes` replaces `10` by `CA`; `shift_rows` moves it to another column; `mix_columns` combines column bytes using `gf_mul`. `add_round_key` then applies K1.

3. The AES loop repeats until round ten, whose conditional skips MixColumns. `state_to_bytes` returns `69C4E0D86A7B0430D8CDB78070B4C55A` for the official vector. Decryption starts with K10 and reverses the transformations to recover P.

4. In SM4, `sm4_encrypt_block(P,K)` delegates to `sm4_process_block`. It unpacks P into X0 through X3 and calls `key_expansion`. The first encryption subkey is `F12186F9`.

5. `round` forms A0=`F002C39E`, applies `sbox` to obtain `18E992B1`, and `linear` to obtain `26D99622`. XOR with X0=`01234567` gives X4=`27FAD345`. The returned window is X1,X2,X3,X4.

6. After thirty-two updates, SM4 serializes X35,X34,X33,X32, producing `681EDF34D206965E86B3E94F536E4246`. The decryption wrapper uses the same recurrence with the subkeys in reverse order and checks the original block is recovered.

11.4 End-to-end trace: the four fault-recovery paths

1. **AES round-nine Piret:** `pdf_aes_lesson` or `run.aes_demo` asks `encrypt_with_trace` for a byte fault. Its observer mutates the real AES state before MixColumns. `key_candidates` receives only the output pair and column; intersections recover a quartet. After four unique groups, `reverse_key_schedule` reconstructs K0. The direct `piret.py` command instead uses the documented round-eight generator and `recover_last_key` to process all groups together.

2. **AES Giraud:** `aes_fault_encrypt` changes one bit before final SubBytes. `aes_candidates` identifies the single changed output byte and tests the inverse-S-box equation for every byte guess. `intersect_candidates` retains shared survivors. Each unique output byte fills its K10 position; reversal and re-encryption follow only after all sixteen positions are unique.

3. **SM4 byte DFA:** `recover_sm4_master_key` asks an output-only oracle for a selected round/byte/mask. `peel_sm4` removes already recovered later rounds, and `sm4_byte_candidates` removes L from the difference. Intersections identify a subkey byte. All sixteen subkey bytes across rounds 31–28 must be unique before `reverse_sm4` returns the master key.

4. **SM4 Giraud-inspired:** the same outer recovery loop runs, but the wrapper supplies `sm4_candidates` through `candidate_filter`. Unknown-bit mode tests all eight bit masks and never forwards the actual simulator mask. Controlled target access at all four rounds is still required; changing the filter does not remove that assumption.

These paths share data structures and target engines, but their observation requirements and allowed fault sets differ. A teaching observer is allowed to show an intermediate value; the recovery API is not automatically entitled to receive it.

11.5 How to use the function reference

Every entry below gives the actual callable signature, its source line range in the appendix, and ordered implementation steps. Methods include `self`, the application or terminal instance on which they operate. Nested callbacks are described after their containing module. The appendix retains decorators, including the cached inverse-matrix decorator, as well as complete parent functions containing the callbacks. Signature wrapping in the typeset listing does not change the one-line argument convention in the source files. Code uses plain monospaced rendering to reduce compilation work; all source text is retained. The contents lists chapters and sections, while individual function headings remain in the body.

11.6 `aes.py`: function-by-function implementation

Read conversion first, then field arithmetic, transformations, key expansion, encryption and inverse operations. Matrix transformations mutate their argument and return None; block functions return immutable bytes. The operation wrapper is the only generic instrumentation seam.

11.6.1 Implementation topics and presentation order

1. **Inputs and constants.** Introduce the sixteen-byte plaintext and key, ten rounds, `SBOX`, inverse table, `RCON` and mixing matrices. Separate editable example inputs from the explicit arguments of the reusable block functions.

2. **State representation.** Follow `validate_block`, `bytes_to_state` and `state_to_bytes`. Explain the column-major rule $j = 4c + r$, using one input byte to show why serialization order matters.

3. **Finite-field arithmetic.** Explain `xtime` before `gf_mul`: shifts, overflow detection, reduction by the AES polynomial and XOR accumulation. Contrast this with ordinary integer multiplication.

4. **Round transformations.** Explain `sub_bytes`, `shift_rows`, `mix_column`, `mix_columns` and `add_round_key`. State which operation changes byte values, moves their positions, mixes a column or incorporates a key. These transformations modify the working matrix in place.

5. **Key expansion.** Follow `rot_word`, `sub_word` and `key_expansion`. Show how four initial words become forty-four words and eleven keys, with a special transformation on every fourth word.

6. **Encryption control flow.** Walk through `aes_encrypt`: initial K0 XOR, nine full rounds, and round ten without MixColumns. Explain how `_operation` surrounds transformations with observation events while keeping fault rules outside the cipher.

7. **Decryption.** Follow `inv_shift_rows`, `inv_sub_bytes`, `inv_mix_columns` and `aes_decrypt`. Show the reversed operation/key order and distinguish decrypting with a known key from recovering an unknown key.

8. **Verification and demonstration.** Explain `main`, compare the official ciphertext with the implementation result and check that decryption recovers the plaintext. A round trip alone does not replace a known-answer test. Direct terminal execution also uses the shared logging wrapper.

Suggested explanation sequence: input bytes → state matrix → expanded keys → round transformations → ciphertext → inverse operations → verification.

11.6.2 `validate_block`

```
def validate_block(data, label="Block"):
```

Source: complete `aes.py` listing in Part 6, lines 101–104.

1. Accept a byte sequence and a human-readable label. Check its length against BLOCK_SIZE, which is sixteen.

2. Raise ValueError on a wrong length; otherwise return None. This validates size, not message padding or a general encryption mode.

11.6.3 `bytes_to_state`

```
def bytes_to_state(data):
```

Source: complete `aes.py` listing in Part 6, lines 107–116.

1. Validate the sixteen input elements and allocate four independent rows.

2. Traverse columns outside rows and assign data[4*column+row] to state[row][column]. Return a fresh mutable matrix, so later transformations do not mutate the caller's bytes.

11.6.4 `state_to_bytes`

```
def state_to_bytes(state):
```

Source: complete `aes.py` listing in Part 6, lines 119–121.

1. Traverse the matrix column by column in exactly the inverse conversion order.
2. Construct and return immutable bytes. For example, matrix entry state[1][2] becomes output byte nine, not byte six.

11.6.5 `xtime`

```
def xtime(a):
```

Source: complete `aes.py` listing in Part 6, lines 129–137.

1. Shift the byte left once to multiply its polynomial by x. Test the original high bit to detect the x^8 overflow.
2. If that bit was set, XOR 0x1B into the shifted result, then mask to eight bits. Thus xtime(0x80) is 0x1B, not the ordinary integer product 0x100.

11.6.6 `gf_mul`

```
def gf_mul(a, b):
```

Source: complete `aes.py` listing in Part 6, lines 140–152.

1. Initialize an XOR accumulator and inspect eight bits of multiplier b. Whenever the low bit is one, XOR the current multiplicand a into the result.
2. Advance a with xtime and shift b right after each iteration. Return one field byte; multiplication by 02 and 03 in MixColumns therefore uses field reduction rather than ordinary multiplication.

11.6.7 `sub_bytes`

```
def sub_bytes(state):
```

Source: complete `aes.py` listing in Part 6, lines 160–168.

1. Visit every state coordinate and use its current byte as an index into SBOX.
2. Overwrite that coordinate with the substituted byte. Return None: this operation changes values in the supplied matrix in place, without changing their positions.

11.6.8 `shift_rows`

```
def shift_rows(state):
```

Source: complete `aes.py` listing in Part 6, lines 171–174.

1. For row indices one through three, split the row after its row index.
2. Concatenate the suffix and prefix to rotate that row left by its index. Row zero stays fixed, and the function updates the matrix in place without returning a new state.

11.6.9 `mix_column`

```
def mix_column(column):
```

Source: complete `aes.py` listing in Part 6, lines 177–189.

1. Accept four original column bytes and initialize an empty output list. For each row of MIX_MATRIX, XOR the four coefficient/byte products computed by gf_mul.
2. Append each resulting byte and return the new four-byte list. Keeping the original column separate prevents an earlier output assignment from contaminating a later equation.

11.6.10 `mix_columns`

```
def mix_columns(state):
```

Source: complete `aes.py` listing in Part 6, lines 192–197.

1. Extract one column into an independent four-element list and call mix_column.
2. Write its four returned bytes back to the same column and repeat for all four columns. The state is updated in place; no printing or fault rule is introduced.

11.6.11 `add_round_key`

```
def add_round_key(state, round_key):
```

Source: complete `aes.py` listing in Part 6, lines 200–208.

1. Read the state and round-key matrices at matching row/column coordinates.
2. XOR each key byte into the state byte in place. Applying the same key twice undoes the operation, and applying it to both members of a differential pair preserves their XOR difference.

11.6.12 `rot_word`

```
def rot_word(word):
```

Source: complete `aes.py` listing in Part 6, lines 216–218.

1. Take a four-byte key-schedule word and slice off its first byte.
2. Append that byte after the other three and return a new list. This rotates key-word bytes, not bits, and leaves the argument unchanged.

11.6.13 `sub_word`

```
def sub_word(word):
```

Source: complete `aes.py` listing in Part 6, lines 221–223.

1. Take each byte of a key-schedule word as an SBOX index.
2. Return a new four-byte list of substituted values. This uses the same forward AES S-box as encryption, not INV_SBOX.

11.6.14 `key_expansion`

```
def key_expansion(key):
```

Source: complete `aes.py` listing in Part 6, lines 226–259.

1. Validate the master key and divide it into four four-byte words. Generate words four through forty-three by XORing the word four places back with a transformed copy of the previous word.
2. On every fourth word, first apply rot_word, sub_word and RCON to the first byte. Group each four consecutive words into a fresh state matrix and return eleven matrices K0 through K10.

11.6.15 `_operation`

```
def _operation(state, number, name, transform, observer):
```

Source: complete `aes.py` listing in Part 6, lines 267–277.

1. Accept a mutable state, round number, operation name, transformation callable and optional observer. If present, notify the observer with a Before event and the live state.
2. Run transform(state), then issue the corresponding After event. The observer can modify the live matrix; the cipher itself defines no coordinate or XOR mask for an attack.

11.6.16 `aes_encrypt`

```
def aes_encrypt(plaintext, key, observer=None):
```

Source: complete `aes.py` listing in Part 6, lines 280–291.

1. Convert the plaintext to a matrix, expand the key and apply the initial AddRoundKey with K0 through _operation.
2. Run rounds one through ten in SubBytes, ShiftRows, optional MixColumns, AddRoundKey order. Skip MixColumns in round ten, serialize the final matrix and return ciphertext bytes without printing.

11.6.17 `inv_sub_bytes`

```
def inv_sub_bytes(state):
```

Source: complete `aes.py` listing in Part 6, lines 294–298.

1. Read each current state byte and index INV_SBOX.
2. Replace the byte in place. The inverse table undoes the forward substitution independently at each coordinate.

11.6.18 `inv_shift_rows`

```
def inv_shift_rows(state):
```

Source: complete `aes.py` listing in Part 6, lines 301–304.

1. Visit rows one through three and split each row before its final row-index bytes.
2. Move those final bytes to the front. These right rotations undo shift_rows while leaving row zero unchanged.

11.6.19 `inv_mix_columns`

```
def inv_mix_columns(state):
```

Source: complete `aes.py` listing in Part 6, lines 307–315.

1. Copy all four original bytes of each column before any assignment.
2. For every output row, XOR products with the corresponding INV_MIX_MATRIX coefficients, using gf_mul, and store the result. The copy ensures every inverse output uses the same original column.

11.6.20 `aes_decrypt`

```
def aes_decrypt(ciphertext, key, observer=None):
```

Source: complete `aes.py` listing in Part 6, lines 318–329.

1. Convert ciphertext to a state, expand the supplied key and remove K10 with AddRoundKey.
2. Iterate key indices nine down to zero, applying InvShiftRows, InvSubBytes and AddRoundKey, followed by InvMixColumns except at index zero. Serialize and return the recovered plaintext; this is ordinary decryption, not a key-recovery attack.

11.6.21 `main`

```
def main():
```

Source: complete `aes.py` listing in Part 6, lines 332–344.

1. Decode configured KEY and PLAINTEXT hexadecimal strings, encrypt once and decrypt the result.
2. Print the key, plaintext, ciphertext and recovered block; raise RuntimeError if the recovered input differs. The main guard calls this routine only when aes.py is executed directly.

11.6.22 Why the AddRoundKey lambdas are needed

`_operation` expects a transformation taking only the current state. An expression such as `lambda s: add_round_key(s, keys[number])` adapts the two-argument key-XOR primitive to that interface. It is invoked immediately during the current loop iteration; it is not stored for later execution. Consequently it uses the intended round key rather than a later loop index.

11.7 `sm4.py`: function-by-function implementation

Read word rotations and the two distinct linear maps, then tau, observation semantics, round recurrence, key recurrence and common block processing. Unlike AES, each round returns a replacement four-word window. Encryption and decryption differ in subkey order.

11.7.1 Implementation topics and presentation order

1. **Inputs, words and constants.** Introduce the sixteen-byte block/key, four big-endian words, thirty-two rounds, SM4 `SBOX`, `FK`, `CK` and word mask. The SM4 S-box is not the AES S-box.

2. **Circular rotation.** Explain `rotl`, its shift/mask operations and how wrapped bits re-enter the word. Follow one 32-bit input through a rotation before introducing the diffusion formulas.

3. **Substitution and diffusion.** Explain `sbox` as four independent byte substitutions. Compare `linear`, using rotations 2, 10, 18 and 24, with `linear_key`, using rotations 13 and 23. These maps serve different recurrences.

4. **Key schedule.** Follow `generate_round_key` and `key_expansion`: XOR the master words with FK, slide a four-word window, incorporate CK and generate rk_0 through rk_{31}.

5. **One data round.** Trace `round`: compute $A = X_{i+1} \oplus X_{i+2} \oplus X_{i+3} \oplus rk_i$, then $B = \tau(A)$, $D = L(B)$ and $X_{i+4} = X_i \oplus D$. Return the updated four-word window.

6. **Observation mechanism.** Explain `observe`: an absent callback preserves the value, None preserves it too, and another returned value replaces it. This is how an external experiment can intervene without embedding an attack model in normal SM4.

7. **Complete encryption and decryption.** Follow `sm4_process_block`, `sm4_encrypt_block` and `sm4_decrypt_block`. Explain thirty-two updates, final word reversal and reverse subkey order for decryption. Word reversal does not reverse the bytes inside each word.

8. **Verification and demonstration.** Use `main` and the official ciphertext to validate the result, then check recovery of the input. Explain that this is a raw-block implementation; message padding and authenticated modes are outside these functions. Terminal logging is handled by the shared wrapper.

Suggested explanation sequence: block → four words → key expansion → XOR → substitution → diffusion → rolling update → final reversal.

11.7.2 `rotl`

```
def rotl(val: int, count: int) -> int:
```

Source: complete `sm4.py` listing in Part 7, lines 71–73.

1. Accept a 32-bit word and a rotation count used by the fixed SM4 transforms. Shift left, mask the overflowing high bits and shift the original word right by 32-count.

2. OR the two pieces to return a circular left rotation. The fixed call sites use counts two, ten, eighteen, twenty-four, thirteen and twenty-three.

11.7.3 `linear`

```
def linear(b: int) -> int:
```

Source: complete `sm4.py` listing in Part 7, lines 76–82.

1. Take the 32-bit substituted data word B. Compute rotations by two, ten, eighteen and twenty-four using rotl.

2. XOR B with all four rotations and return L(B). This map is XOR-linear, which permits differential inversion in the SM4 attacks.

11.7.4 `linear_key`

```
def linear_key(b: int) -> int:
```

Source: complete `sm4.py` listing in Part 7, lines 85–87.

1. Take a word from the key-schedule substitution.
2. XOR it with rotations thirteen and twenty-three and return L-prime. The key schedule deliberately uses a different diffusion map from the data rounds.

11.7.5 `sbox`

```
def sbox(a: int) -> int:
```

Source: complete `sm4.py` listing in Part 7, lines 90–94.

1. Pack the input word into four big-endian bytes using struct.pack.
2. Substitute each byte through the SM4 SBOX and unpack the four bytes back into a 32-bit word. This implements tau without changing byte order.

11.7.6 `observe`

```
def observe(observer, number, operation, value):
```

Source: complete `sm4.py` listing in Part 7, lines 102–111.

1. If no observer exists, immediately return the original intermediate value. Otherwise call observer(number, operation, value).
2. Retain the original value when the callback returns None, or return the replacement otherwise. Unlike AES matrix mutation, SM4 callbacks replace immutable integer, tuple or bytes values through this return channel.

11.7.7 `round`

```
def round(x: list[int], rk: int, observer=None, round_index=0):
```

Source: complete `sm4.py` listing in Part 7, lines 114–124.

1. Accept four rolling state words and the round subkey. Form the XOR of the three newest words and the subkey, then pass it through the Sbox input observation event.
2. Apply sbox and linear with their observation events, XOR the result with the oldest word, and return the three newer words followed by this new word. The input list is not modified in place.

11.7.8 `generate_round_key`

```
def generate_round_key(k: list[int], round: int) -> int:
```

Source: complete `sm4.py` listing in Part 7, lines 127–133.

1. Accept four consecutive schedule words and a zero-based round index. XOR the three newest words with CK at that index.
2. Apply sbox and linear_key, then XOR the oldest schedule word. Return one new 32-bit schedule word, which is the encryption subkey for that round.

11.7.9 key_expansion

```
def key_expansion(key: bytes) -> list[int]:
```

Source: complete `sm4.py` listing in Part 7, lines 141–154.

1. Reject keys whose length is not sixteen and unpack the master key as four big-endian words. XOR each with its FK constant to initialize the schedule.
2. Generate and append thirty-two new words with generate_round_key on the sliding four-word window. Return only those thirty-two new words, excluding the initial whitened master-key words.

11.7.10 sm4_process_block

```
def sm4_process_block(block: bytes, key: bytes, encrypt: bool, observer=None) -> bytes:
```

Source: complete `sm4.py` listing in Part 7, lines 162–178.

1. Validate the block, unpack four words and expand the key. For each of thirty-two iterations, select the forward subkey index for encryption or the reversed index for decryption.
2. Observe the input state, call round, observe the output state, then reverse the four final words and pack them into bytes. Return the final observer result; reversing word order is not reversing the bytes inside each word.

11.7.11 sm4_encrypt_block

```
def sm4_encrypt_block(x: bytes, key: bytes, observer=None) -> bytes:
```

Source: complete `sm4.py` listing in Part 7, lines 181–183.

1. Accept a raw sixteen-byte block, master key and optional observer.
2. Delegate to sm4_process_block with encrypt=True and return its bytes. This wrapper supplies no attack parameters.

11.7.12 sm4_decrypt_block

```
def sm4_decrypt_block(x: bytes, key: bytes, observer=None) -> bytes:
```

Source: complete `sm4.py` listing in Part 7, lines 186–188.

1. Accept the ciphertext block, master key and optional observer.
2. Delegate to sm4_process_block with encrypt=False, so the same data recurrence runs with reversed round keys. No inverse S-box or separate inverse data-round implementation is needed.

11.7.13 main

```
def main():
```

Source: complete `sm4.py` listing in Part 7, lines 196–207.

1. Decode the configured official hexadecimal vector and call the normal encrypt and decrypt wrappers.
2. Print the three blocks and check the recovered block equals the plaintext. This verifies a round trip; known-answer tests additionally compare against the official ciphertext.

Chapter 12

Attack Functions: Piret and Giraud

12.1 `piret.py`: function-by-function implementation

Read public differential helpers, external target adapters, SM4 inversion/filtering/peeling, AES quartet filtering and reverse schedules before terminal formatting. Keep the legacy known-key relation lesson separate from the ciphertext-only recovery paths.

12.1.1 Implementation topics and presentation order

1. **Experimental roles.** Separate the normal cipher, key-owning simulated target, ciphertext-based solver and teaching display. Explain why showing a hidden state for learning does not permit giving it to the solver.

2. **Differential helpers.** Explain `difference`, `clone`, `gf_multiply` and `predicted_mixcolumns_differen` For a row-zero fault, derive the four-byte pattern $(2e, e, e, 3e)$ in the AES field.

3. **AES fault injection.** Follow `encrypt_with_trace` and its observer before MixColumns. Compare round nine, which reaches one quartet, with round eight, which reaches all four groups. `trace_round8` is a compatibility name that also accepts an explicit round-nine request.

4. **AES candidate construction.** Explain `output_positions` and `key_candidates`: undo the final key XOR/S-box under each guess, match a shared error row and magnitude, and retain correlated four-byte tuples. Independent byte guesses are insufficient.

5. **AES complete key recovery.** Explain per-column intersections in the round-nine lesson and `recover_last_key` for the round-eight path. Once K10 is unique, follow `reverse_key_schedule` to K0 and verify by re-encryption. Keep the known-key `relation_demo` distinct from unknown-key reconstruction.

6. **SM4 controlled-byte model.** Follow `sm4_fault_encrypt`, `inverse_rows`, `inverse_linear` and `sm4_byte_candidates`. Undo L, test S-box input candidates using the known byte mask, and convert them to subkey-byte guesses using the observable words.

7. **SM4 multi-round recovery.** Follow `peel_sm4` and `recover_sm4_master_key`. Recover rk_{31} through rk_{28} using fresh faults at each target round and only previously recovered later keys. Explain the additional selectable-round fault assumption.

8. **Master-key reconstruction and output.** Follow `reverse_sm4`, remove FK and assemble the original master key. Earlier SM4 subkeys are derived by the reversed schedule, whereas the final four came from DFA. Explain `Terminal`, `main`, red differences, step mode and shared TXT/HTML logging.

9. **Ambiguity and failure.** Distinguish an incompatible empty intersection from multiple compatible survivors. Neither case permits filling missing bytes from the teaching key or claiming successful complete recovery.

Suggested explanation sequence: fault location → observed difference → candidate equations → intersections → recovered subkeys → inverse schedule → verified master key.

12.1.2 `difference`

```
def difference(a, b):
```

Source: complete `piret.py` listing in Part 8, lines 65–67.

1. Accept two AES matrices with matching coordinates.
2. Return a new matrix containing the coordinate-wise XOR. The inputs remain unchanged, allowing a teacher to compare saved states safely.

12.1.3 `clone`

```
def clone(state):
```

Source: complete `piret.py` listing in Part 8, lines 70–72.

1. Accept a nested row-list state.
2. Copy every row with slicing and return the new outer list. Copying only the outer list would leave shared rows and would not protect a saved snapshot against later in-place changes.

12.1.4 `gf_multiply`

```
def gf_multiply(a, b):
```

Source: complete `piret.py` listing in Part 8, lines 75–81.

1. Accept two field bytes under the historical attack-helper name.
2. Return aes.gf_mul(a,b). Sharing the cipher's field arithmetic avoids a second implementation with a different reduction polynomial.

12.1.5 `predicted_mixcolumns_difference`

```
def predicted_mixcolumns_difference(row, fault):
```

Source: complete `piret.py` listing in Part 8, lines 84–89.

1. Accept the faulty input row and a nonzero byte difference.
2. Multiply that scalar error by the matching column of MIX_MATRIX using gf_multiply. Return four predicted differences; row zero gives the ordered pattern [2e,e,e,3e].

12.1.6 `reverse_last_round_difference`

```
def reverse_last_round_difference(correct_ciphertext, faulty_ciphertext, last_round_key,):
```

Source: complete `piret.py` listing in Part 8, lines 92–118.

1. Accept correct/faulty ciphertexts and an already known K10 matrix, then convert the two blocks to state matrices. Remove final AddRoundKey, undo ShiftRows and apply INV_SBOX.
2. Return the XOR of the resulting states. This verifies a known-key differential relation; because K10 is supplied, it is not the unknown-key candidate solver.

12.1.7 `encrypt_with_trace`

```
def encrypt_with_trace(plaintext, key, row=0, column=0, mask=0, fault_round=8):
```

Source: complete `piret.py` listing in Part 8, lines 126–144.

1. Validate coordinates, byte mask and the choice of round eight or nine. Install a local observer that changes one state byte before the selected MixColumns event.

2. Run aes.aes_encrypt once with this observer and return its ciphertext plus immutable after-operation snapshots. The target adapter sees the simulation key, but the candidate solver is called separately with output observations.

12.1.8 `encrypt_with_trace.observer`

```
def observer(number, operation, state):
```

Source: complete `piret.py` listing in Part 8, lines 136–141.

1. Match the requested round and Before MixColumns event; XOR the selected live state byte. Save a serialized Fault point and after-operation snapshots. Return values are unused because AES observers mutate the matrix.

12.1.9 `sm4_fault_encrypt`

```
def sm4_fault_encrypt(plaintext, key, byte=None, mask=0, fault_round=31):
```

Source: complete `piret.py` listing in Part 8, lines 147–169.

1. Validate the selected SM4 byte, mask and round; expand the target key only for the diagnostic trace. Install a callback at the Sbox input event that shifts the byte mask into the chosen big-endian position.

2. Run sm4.sm4_encrypt_block with this observer and return ciphertext and per-round A, B, linear, new-word and subkey trace fields. Full-recovery oracle closures discard the trace and expose only the ciphertext to the solver.

12.1.10 `sm4_fault_encrypt.observer`

```
def observer(number, operation, value):
```

Source: complete `piret.py` listing in Part 8, lines 161–166.

1. At the selected Sbox input, XOR the shifted mask into the integer value. Store selected scalar event values in the diagnostic round dictionary and return the possibly replaced value to the SM4 engine.

12.1.11 `trace_round8`

```
def trace_round8(plaintext, key, row=0, column=0, mask=0, fault_round=8):
```

Source: complete `piret.py` listing in Part 8, lines 172–178.

1. Accept the same explicit parameters as encrypt_with_trace, including fault_round.

2. Delegate directly and return its ciphertext/trace pair. The historical function name does not force round eight: callers can explicitly request round nine.

12.1.12 `encrypt`

```
def encrypt(plaintext, key, inject_fault=False):
```

Source: complete `piret.py` listing in Part 8, lines 181–192.

1. Read the legacy fault-coordinate globals and choose their mask only when inject_fault is true. Run the common AES target adapter at round nine.

2. Translate selected trace events into named matrix snapshots and return ciphertext, snapshots and expanded key matrices. This compatibility interface supports the known-key relation lesson, not a ciphertext-only recovery claim.

12.1.13 `aes_target_encrypt`

```
def aes_target_encrypt(plaintext, key):
```

Source: complete `piret.py` listing in Part 8, lines 195–197.

1. Accept the plaintext and target key.

2. Call aes.aes_encrypt without an observer and return the clean ciphertext. This supplies an ordinary baseline without changing global cipher settings.

12.1.14 `inverse_rows`

```
def inverse_rows():
```

Source: complete `piret.py` listing in Part 8, lines 208–218.

1. Construct the binary matrix of SM4 L by applying linear to each of the thirty-two unit-bit words. Augment its rows with an identity matrix.

2. Perform binary Gaussian elimination using row swaps and XOR, then return the rows of the inverse matrix. The lru_cache decorator stores this fixed result so later calls do not repeat elimination.

12.1.15 `inverse_linear`

```
def inverse_linear(value):
```

Source: complete `piret.py` listing in Part 8, lines 221–222.

1. Accept a 32-bit output difference.

2. For each cached inverse-matrix row, compute the parity of row AND value and place that parity into its output bit. Return L-inverse(value), using binary arithmetic rather than a guessed inverse rotation.

12.1.16 `sm4_byte_candidates`

```
def sm4_byte_candidates(correct, faulty, byte, mask):
```

Source: complete `piret.py` listing in Part 8, lines 225–239.

1. Validate the two output blocks, byte index and known mask; require the other three words to agree. Invert the first-word XOR through inverse_linear and require support in exactly the requested S-box byte.

2. Enumerate all input bytes a satisfying the S-box differential, convert them to key-byte candidates by XOR with the observable three-word combination, and return candidates, input guesses and the 32-bit inverse-L difference.

12.1.17 `reverse_sm4`

```
def reverse_sm4(final_keys, observer=None):
```

Source: complete `piret.py` listing in Part 8, lines 242–252.

1. Require four consecutive recovered subkeys in rk28,rk29,rk30,rk31 order and place them at schedule indices thirty-two through thirty-five.

2. Iterate backward using CK, sbox and linear_key to derive earlier schedule words. Undo the four FK constants and pack the resulting master words; optionally report each recovered schedule word to an observer.

12.1.18 `peel_sm4`

```
def peel_sm4(ciphertext, target_round, recovered_keys):
```

Source: complete `piret.py` listing in Part 8, lines 255–271.

1. Validate the requested round in twenty-eight through thirty-one and reverse ciphertext word order into a rolling window.

2. Undo every later round using only entries already present in recovered_keys; reject a missing key. Return a reversed-word view at the target round, allowing reuse of the same last-round candidate equations.

12.1.19 `recover_sm4_master_key`

```
def recover_sm4_master_key(correct, fault_oracle, masks=(1, 2, 3, 4, 5, 6, 7, 8), start_byte=0,
↪  candidate_filter=None, observer=None):
```

Source: complete `piret.py` listing in Part 8, lines 274–318.

1. Validate distinct nonzero masks and iterate target rounds thirty-one down to twenty-eight. For each target byte, query the output-only oracle, peel correct and faulty outputs and apply the known-byte filter or the supplied replacement filter.

2. Intersect candidates, emit history events and stop with no master key when ambiguity remains. Once all four final subkeys are uniquely recovered, call reverse_sm4 and return the master key, recovered subkeys, counts and history.

12.1.20 `output_positions`

```
def output_positions(column):
```

Source: complete `piret.py` listing in Part 8, lines 322–327.

1. Validate an AES column index in zero through three.

2. Return indices 4*((column-row) modulo 4)+row for rows zero through three. This undoes the final ShiftRows permutation for a quartet while retaining the original row order used by the mixing equation.

12.1.21 `key_candidates`

```
def key_candidates(correct, faulty, column):
```

Source: complete `piret.py` listing in Part 8, lines 330–355.

1. For each of the four mapped ciphertext bytes, test all 256 K10 byte guesses and group them by the inverse-S-box difference they produce.
2. Enumerate one shared faulty row and nonzero error, use the corresponding four Mix-Columns coefficients to select compatible lists, and take their Cartesian product. Return a set of correlated four-byte tuples rather than independent byte sets.

12.1.22 `recover_last_key`

```
def recover_last_key(correct, faulty_outputs, observer=None):
```

Source: complete `piret.py` listing in Part 8, lines 358–386.

1. Process round-eight faulty outputs, requiring every ciphertext byte to change under that specific model. Use key_candidates for each of four output groups and intersect its tuples across observations.
2. Reject empty intersections; when every group is unique, map all tuple bytes into a sixteen-byte K10 and return it with the observation count. If observations end first, return None instead of a guessed key.

12.1.23 `reverse_key_schedule`

```
def reverse_key_schedule(last_key, observer=None):
```

Source: complete `piret.py` listing in Part 8, lines 394–415.

1. Place the recovered sixteen K10 bytes into words forty through forty-three. For descending indices, copy the previous word, applying RotWord, forward SubWord and Rcon only when the index is a multiple of four.
2. XOR this transformed word with the current word to obtain the word four positions earlier. Optionally notify a presentation observer, then concatenate the first four words to return K0.

12.1.24 `show_state`

```
def show_state(title, state):
```

Source: complete `piret.py` listing in Part 8, lines 423–428.

1. Accept a title and a row-indexed AES matrix.
2. Print a heading and format each matrix row as two-digit hexadecimal bytes. This is presentation only and returns no cryptographic result.

12.1.25 `show_difference`

```
def show_difference(title, a, b):
```

Source: complete `piret.py` listing in Part 8, lines 431–433.

1. Accept a title and two saved matrices.
2. Compute their difference and pass it to show_state. Neither input is changed, and no candidate filtering occurs.

12.1.26 `Terminal.__init__`

```
def __init__(self, color="auto", step=False):
```

Source: complete `piret.py` listing in Part 8, lines 438–447.

1. Resolve color mode from the explicit option, terminal detection and NO_COLOR environment variable, and store the step-mode flag.
2. When color is active on Windows, enable virtual-terminal processing through the console API. Presentation configuration does not change target bytes or solver equations.

12.1.27 `Terminal.paint`

```
def paint(self, text, code="36"):
```

Source: complete `piret.py` listing in Part 8, lines 449–451.

1. Accept text and an ANSI style code.
2. Return the styled string plus reset sequence when color is enabled, or plain text otherwise. Byte values are formatted elsewhere before this method receives them.

12.1.28 `Terminal.section`

```
def section(self, title):
```

Source: complete `piret.py` listing in Part 8, lines 453–456.

1. Accept a teaching-section title.
2. Print a separator and the title through paint. This creates a visual boundary without advancing an algorithm or changing candidate sets.

12.1.29 `Terminal.pause`

```
def pause(self):
```

Source: complete `piret.py` listing in Part 8, lines 458–461.

1. Inspect the stored step-mode flag.
2. Call input only when stepping is enabled; otherwise return immediately. Batch and default execution therefore remain noninteractive.

12.1.30 `Terminal.compare`

```
def compare(self, correct, faulty):
```

Source: complete `piret.py` listing in Part 8, lines 463–477.

1. Accept two serialized states, compute their XOR, and display the bytes in AES row order using index 4*column+row.
2. Color changed faulty bytes and nonzero differences red, then print changed-byte positions and Hamming distance. These metrics describe the given pair rather than a recovery success probability.

12.1.31 `relation_demo`

```
def relation_demo():
```

Source: complete `piret.py` listing in Part 8, lines 485–585.

1. Decode legacy constants, validate the fault descriptor and obtain clean and faulted round-nine snapshots with encrypt.

2. Compare observed MixColumns differences with the predicted pattern, then undo the final round using the known K10 and assert the differential relation. Explicitly report that the key was known; this routine does not recover an unknown key.

12.1.32 `main`

```
def main(argv=None):
```

Source: complete `piret.py` listing in Part 8, lines 593–715.

1. Parse options, load the target configuration and route SM4 requests to its controlled-byte demonstration. For AES, obtain the clean trace and generate reproducible round-eight faulty outputs through a local generator.

2. Call recover_last_key using only ciphertexts, validate the recovered differential equations and reverse the schedule. Verify by re-encryption and final simulation ground truth; return zero on success or two on unresolved ambiguity.

12.1.33 `main.observations`

```
def observations():
```

Source: complete `piret.py` listing in Part 8, lines 659–666.

1. Iterate the reproducible fault-mask sequence, run the target adapter, display selected teaching comparisons and yield only each faulty ciphertext. Lazy yielding lets the solver stop generating faults once all groups are unique.

12.1.34 `main.report`

```
def report(index, column, before, fresh, survivors):
```

Source: complete `piret.py` listing in Part 8, lines 668–675.

1. Receive the observation index, group, previous count, fresh count and surviving tuples from recover_last_key. Print candidate evolution and representative tuples; do not supply any new candidate or key byte.

12.1.35 `main.schedule_report`

```
def schedule_report(index, current, transformed, previous):
```

Source: complete `piret.py` listing in Part 8, lines 700–703.

1. Receive the current expanded word, transformed predecessor and recovered earlier word. Format the reverse-schedule XOR equation and optionally pause on a key-schedule core boundary.

12.2 `giraud.py`: function-by-function implementation

Read one-bit validation, target adapters, per-pair candidate tests, intersections and orchestration. Exact-mask knowledge is an optional stronger model. The shared SM4 engine is reused through a candidate-filter callback, not by importing hidden key material.

12.2.1 Implementation topics and presentation order

1. **Single-bit assumption.** Introduce `SINGLE_BITS`, `validate_mask` and `masks_for`. A valid fault flips exactly one input bit. Unknown-bit mode knows the error belongs to the eight-mask set; known-bit mode receives the exact mask.

2. **AES target adapter.** Follow `aes_fault_encrypt`: intervene before final-round SubBytes, map a flat byte to matrix coordinates and save snapshots. No later MixColumns follows this point, so the fault reaches one output byte after ShiftRows.

3. **AES inverse-S-box test.** Explain `aes_candidates`: for every guessed final-key byte, evaluate $S^{-1}(C_j \oplus k) \oplus S^{-1}(F_j \oplus k)$. Require a permitted one-bit input difference, not necessarily a one-bit ciphertext XOR.

4. **Intersections and AES key assembly.** Explain `intersect_candidates` and `recover_byte`. Repeat observations at the same output position, assemble all sixteen unique K10 bytes in output order, and reverse the AES schedule. More than one survivor means the byte remains unresolved.

5. **SM4 single-bit adaptation.** Follow `sm4_fault_encrypt` and `sm4_candidates`. Undo L before the S-box differential test, identify its affected byte, enumerate compatible inputs and convert them into subkey-byte candidates.

6. **Filter knowledge.** Explain the nested `candidate_filter` inside `recover_sm4_master_key`. It forwards the simulator mask only in known-bit mode. Unknown-bit filtering must test all eight possible bit errors even if the teaching display reveals which bit was actually injected.

7. **SM4 full recovery.** Explain reuse of the shared DFA engine: recover rk_{31}, undo recovered later rounds, then recover rk_{30}, rk_{29} and rk_{28}. Reverse the schedule and remove FK to recover the master key. This is a Giraud-inspired SM4 model, not the unchanged AES attack.

8. **Terminal orchestration.** Follow `main`, its output-only oracle and display callbacks. Explain candidate equations, unique-byte notifications, restored views, inverse-schedule steps and final re-encryption. Compare `-known-bit`, `-step` and `-color`; logs mirror the execution to TXT and colored HTML.

9. **Verification and scope.** Check full recovery only after uniqueness and compare against the normal cipher output. Distinguish the controlled software experiment from uncharacterized physical glitches on STM32F303.

Suggested explanation sequence: one-bit fault → allowed error set → inverse nonlinear test → candidate intersection → full subkey recovery → master-key verification.

12.2.2 `validate_mask`

```
def validate_mask(mask):
```

Source: complete `giraud.py` listing in Part 9, lines 21–24.

1. Accept an integer mask and test membership in SINGLE_BITS.

2. Raise ValueError unless exactly one of the eight byte bits is set. This excludes zero, multi-bit byte errors and masks wider than one byte.

12.2.3 masks_for

```
def masks_for(first, maximum):
```

Source: complete `giraud.py` listing in Part 9, lines 27–32.

1. Validate the initial mask and a maximum observation count from one through eight.

2. Put the chosen mask first, then append the other single-bit masks in their fixed order and truncate the list. Return distinct faults; repeated identical observations would not add an independent constraint.

12.2.4 aes_fault_encrypt

```
def aes_fault_encrypt(plaintext, key, byte=0, mask=None):
```

Source: complete `giraud.py` listing in Part 9, lines 35–58.

1. Validate the target byte and optional one-bit mask, then attach an observer before AES round-ten SubBytes. Translate the flat byte index to row and column and mutate that state byte only when a mask was supplied.

2. Save clean/faulted injection snapshots and subsequent final-round states, execute the real AES engine and return ciphertext plus trace. A None mask leaves a clean baseline for comparison.

12.2.5 aes_fault_encrypt.observer

```
def observer(number, operation, state):
```

Source: complete `giraud.py` listing in Part 9, lines 47–55.

1. Record the clean round-ten SubBytes input, optionally mutate one bit and record the faulted input. Record later final-round events, naming the final AddRoundKey snapshot Ciphertext.

12.2.6 sm4_fault_encrypt

```
def sm4_fault_encrypt(plaintext, key, byte=None, mask=0, fault_round=31):
```

Source: complete `giraud.py` listing in Part 9, lines 61–66.

1. When a target byte is supplied, require a valid one-bit mask.

2. Delegate to piret.sm4_fault_encrypt with the selected round. This reuses the same SM4 engine while constraining the experiment to the single-bit model.

12.2.7 recover_sm4_master_key

```
def recover_sm4_master_key(correct, fault_oracle, first_mask=1, maximum=8, known_bit=False,
↪  start_byte=0, observer=None):
```

Source: complete `giraud.py` listing in Part 9, lines 69–77.

1. Build a local candidate filter that passes the exact mask onward only when known_bit is true. Generate distinct single-bit masks with masks_for.

2. Call the shared multi-round SM4 recovery engine with this filter, start-byte order and optional observer. Return its complete or ambiguous report without obtaining reference subkeys.

12.2.8 recover_sm4_master_key.candidate_filter

```
def candidate_filter(clean, faulty, simulator_mask):
```

Source: complete `giraud.py` listing in Part 9, lines 73–75.

1. Accept peeled output views and the simulator mask. Pass that mask to sm4_candidates only in known-bit mode; pass None otherwise, deliberately preserving unknown-bit uncertainty.

12.2.9 aes_candidates

```
def aes_candidates(correct, faulty, known_mask=None):
```

Source: complete `giraud.py` listing in Part 9, lines 80–106.

1. Validate both blocks and any known mask, then require exactly one changed ciphertext byte. For each K10 byte guess, remove its XOR and invert the S-box on both observed values.

2. Keep the guess if the input difference is any single-bit mask, or equals the exact known mask in the stronger model. Return the changed output index and surviving byte values.

12.2.10 sm4_candidates

```
def sm4_candidates(correct, faulty, known_mask=None):
```

Source: complete `giraud.py` listing in Part 9, lines 109–151.

1. Require three unchanged output words in the current final/peeled view, invert L on the changed first word and locate the single changed S-box byte.

2. Enumerate input bytes satisfying at least one permitted one-bit differential, then XOR the observable three-word combination to obtain key-byte candidates. Return the byte index, candidate set, input set and inverse-L difference.

12.2.11 intersect_candidates

```
def intersect_candidates(previous, current):
```

Source: complete `giraud.py` listing in Part 9, lines 154–163.

1. Treat None as the first observation and copy the current candidate set; otherwise intersect the previous and current sets.

2. Reject an empty result as an incompatible observation/model pair. Return all survivors, leaving ambiguity explicit instead of selecting an arbitrary byte.

12.2.12 `recover_byte`

```
def recover_byte(algorithm, observations):
```

Source: complete `giraud.py` listing in Part 9, lines 166–183.

1. Choose aes_candidates or sm4_candidates from the algorithm name, then iterate supplied ciphertext observations and any optional known mask.

2. Require all observations to concern the same output/subkey byte and intersect their sets. Return the position and survivors; an empty observation sequence returns None,None and this helper does not simulate a target.

12.2.13 `main`

```
def main(argv=None):
```

Source: complete `giraud.py` listing in Part 9, lines 186–286.

1. Parse algorithm, bit-knowledge, observation and display settings and decode the target configuration. For AES, fault each input byte, locate its final output byte and intersect candidate sets until unique or exhausted.

2. Reverse K10 only after all bytes are unique; for SM4, use the output-only oracle and shared four-round recovery wrapper. Print recovered keys and verify them; return a nonzero status if any selected algorithm remains ambiguous.

12.2.14 `main.oracle`

```
def oracle(number, byte, mask):
```

Source: complete `giraud.py` listing in Part 9, lines 232–233.

1. Capture the simulation plaintext and key, request the selected SM4 round/byte/mask fault and return only the output bytes. Keep the diagnostic trace outside the solver interface.

12.2.15 `main.show`

```
def show(event):
```

Source: complete `giraud.py` listing in Part 9, lines 235–241.

1. Receive one completed SM4 candidate-history event and print raw outputs, already recovered later subkeys, peeled views and survivor counts. Optional pausing affects presentation only.

Chapter 13

Terminal and GUI Functions

13.1 `run.py`: function-by-function implementation

This module assembles the normal cipher traces and attack functions into terminal lessons. Compatibility wrappers delegate to piret rather than copying equations. The combined main returns success only when each selected full-recovery lesson succeeds.

13.1.1 `block`

```
def block(value):
```

Source: complete `run.py` listing in Part 10, lines 10–14.

1. Decode one hexadecimal configuration string with bytes.fromhex.
2. Require sixteen bytes and return them, or raise ValueError. This is a raw-block parser, not a conversion of arbitrary text with padding.

13.1.2 `LessonTerminal.changed`

```
def changed(self, original, faulty):
```

Source: complete `run.py` listing in Part 10, lines 18–19.

1. Compare each byte of the displayed faulty value with the byte at the same position in the original.
2. Format the byte in hexadecimal, using inherited paint with red only when it differs, and return one joined display line. The byte arrays themselves remain untouched.

13.1.3 `LessonTerminal.pair`

```
def pair(self, correct, faulty):
```

Source: complete `run.py` listing in Part 10, lines 21–25.

1. Print clean bytes and a red-highlighted faulty line generated by changed.
2. XOR the two sequences and display the difference relative to an all-zero sequence. This flat view is also suitable for SM4 word-order outputs.

13.1.4 aes_demo

```
def aes_demo(terminal, limit, seed):
```

Source: complete `run.py` listing in Part 10, lines 28–96.

1. Read the configured AES key and plaintext, obtain the clean round-nine trace and print normal operations and keys. For each column, inject distinct reproducible round-nine faults and intersect key_candidates quartets.

2. Assemble all unique groups into K10, show inverse schedule steps and verify K0 by re-encryption and ground truth. Return False on ambiguity and True after successful full recovery.

13.1.5 aes_demo.reverse_step

```
def reverse_step(index, current, transformed, recovered):
```

Source: complete `run.py` listing in Part 10, lines 87–90.

1. Receive one AES schedule-reversal event and print the current word XOR transformed predecessor equals the recovered earlier word. This is an observer of reversal, not an alternate reversal algorithm.

13.1.6 inverse_rows

```
def inverse_rows():
```

Source: complete `run.py` listing in Part 10, lines 99–101.

1. Accept no arguments in this compatibility adapter.

2. Return piret.inverse_rows, preserving the shared cached inversion of the SM4 data diffusion matrix.

13.1.7 inverse_linear

```
def inverse_linear(value):
```

Source: complete `run.py` listing in Part 10, lines 104–106.

1. Accept one SM4 word difference.

2. Return piret.inverse_linear(value). The terminal module introduces no second inverse-L implementation.

13.1.8 sm4_encrypt

```
def sm4_encrypt(plaintext, key, byte=None, mask=0, fault_round=31):
```

Source: complete `run.py` listing in Part 10, lines 109–111.

1. Accept an explicit plaintext/key pair and optional byte, mask and fault round.

2. Delegate to piret.sm4_fault_encrypt and return ciphertext plus diagnostic trace. No fault byte means a normal observed encryption.

13.1.9 sm4_byte_candidates

```
def sm4_byte_candidates(correct, faulty, byte, mask):
```

Source: complete `run.py` listing in Part 10, lines 114–116.

1. Accept a correct/faulty view and known byte/mask descriptor.
2. Return the shared piret.sm4_byte_candidates result. This wrapper keeps older callers compatible without duplicating candidate mathematics.

13.1.10 reverse_sm4

```
def reverse_sm4(final_keys, observer=None):
```

Source: complete `run.py` listing in Part 10, lines 119–121.

1. Accept four consecutive final subkeys and an optional schedule observer.
2. Delegate to piret.reverse_sm4 and return its recovered master key. Ordering and completeness validation belong to that shared implementation.

13.1.11 sm4_demo

```
def sm4_demo(terminal, limit, start_byte=0):
```

Source: complete `run.py` listing in Part 10, lines 124–159.

1. Read the configured SM4 target, print a clean trace and define an oracle returning only faulty ciphertexts at the requested round.
2. Request full recovery using known masks one through the configured limit, display each observation and peeled view, and verify the recovered key. Return False on incomplete recovery and True on success.

13.1.12 sm4_demo.oracle

```
def oracle(number, byte, mask):
```

Source: complete `run.py` listing in Part 10, lines 136–137.

1. Capture target plaintext/key, run sm4_encrypt for the requested round, byte and mask and return only ciphertext. The engine may receive a secret key internally, but the recovery solver does not.

13.1.13 sm4_demo.report

```
def report(event):
```

Source: complete `run.py` listing in Part 10, lines 139–146.

1. Receive an SM4 history dictionary and display the actual and peeled ciphertext comparisons, inverse-L difference and survivor values. It neither reads reference subkeys nor changes candidate intersections.

13.1.14 `main`

```
def main(argv=None):
```

Source: complete `run.py` listing in Part 10, lines 162–185.

1. Parse algorithm selection, stepping, colors, observation limit and seed; validate all configured raw blocks.
2. Construct the lesson terminal and execute the selected AES and/or SM4 demonstration. Return zero only when every requested lesson succeeds; the module guard also handles interruption without claiming success.

13.2 `gui.py`: function-by-function implementation

The first functions build immutable lessons. CryptoLab then manages Tk widgets, captured settings, worker execution, queue delivery, timeline rendering, playback and JSON export. Only the Tk thread updates widgets; cryptographic work is performed by the worker.

13.2.1 `parse_block`

```
def parse_block(value):
```

Source: complete `gui.py` listing in Part 11, lines 21–25.

1. Decode the text field as hexadecimal bytes.
2. Require exactly sixteen bytes and return the block; malformed hex or a wrong length raises ValueError for the GUI to report.

13.2.2 `snapshot`

```
def snapshot(number, title, explanation, state, detail="", reference=None):
```

Source: complete `gui.py` listing in Part 11, lines 28–29.

1. Accept a round, title, explanation, state, optional detail and optional correct reference.
2. Convert state/reference bytes to hexadecimal strings and return a dictionary. Immutable serialized states protect the timeline from subsequent in-place cipher changes and can be exported as JSON.

13.2.3 `aes_trace`

```
def aes_trace(block, key, decrypt=False, fault=None):
```

Source: complete `gui.py` listing in Part 11, lines 32–57.

1. Start a timeline with the input block and expand keys for teaching details. In normal mode, use an after-operation observer on the actual AES encrypt/decrypt function.
2. In fault mode, use the Piret target adapter at the specified round and map its snapshots into the same schema. Return output bytes and steps; refuse faulted decryption because that lesson models encryption faults.

13.2.4 aes_trace.observer

```
def observer(number, operation, state):
```

Source: complete `gui.py` listing in Part 11, lines 49–53.

1. Ignore Before events and translate each After event into a snapshot with an operation description. For AddRoundKey, include the corresponding teaching round key in the detail text.

13.2.5 sm4_trace

```
def sm4_trace(block, key, decrypt=False):
```

Source: complete `gui.py` listing in Part 11, lines 60–86.

1. Start from the input block and attach an observer to the normal SM4 engine. Record rolling-state inputs and the scalar A, tau(A), linear and new-word events.

2. Add explanatory snapshots and final word reversal, using reversed subkey indices for decryption. Return the engine output and timeline; scalar substitution stages do not yet replace the displayed rolling state.

13.2.6 sm4_trace.observer

```
def observer(index, operation, value):
```

Source: complete `gui.py` listing in Part 11, lines 65–82.

1. Track State input/output, collect scalar S-box intermediates and add explanatory snapshots. Use nonlocal current to retain the rolling state until its update, and append final word reversal when the Ciphertext event arrives.

13.2.7 positions

```
def positions(column):
```

Source: complete `gui.py` listing in Part 11, lines 89–91.

1. Accept one AES column index.

2. Delegate to piret.output_positions through the console module and return the final ShiftRows byte mapping used by the presentation.

13.2.8 candidates

```
def candidates(correct, faulty, column):
```

Source: complete `gui.py` listing in Part 11, lines 94–96.

1. Accept correct/faulty AES ciphertexts and an output group.

2. Delegate to piret.key_candidates and return its correlated quartet set. Mathematical filtering stays in the attack module.

13.2.9 `reverse_schedule`

```
def reverse_schedule(last_key):
```

Source: complete `gui.py` listing in Part 11, lines 99–100.

1. Accept a recovered AES K10.
2. Delegate to piret.reverse_key_schedule and return the master key without reading GUI key fields.

13.2.10 `recover_key`

```
def recover_key(correct, observations):
```

Source: complete `gui.py` listing in Part 11, lines 103–111.

1. Accept a correct ciphertext and round-eight faulty-output sequence, and install a local record callback.
2. Call piret.recover_last_key and translate observer events into GUI history tuples. Return recovered K10 or None, the history and the number of consumed faults.

13.2.11 `recover_key.record`

```
def record(index, column, before, fresh, survivors):
```

Source: complete `gui.py` listing in Part 11, lines 107–108.

1. Translate the round-eight solver notification into a GUI history tuple containing observation, group, fresh candidates and remaining survivors. This records counts after filtering rather than performing the filtering.

13.2.12 `piret_lesson`

```
def piret_lesson(block, key, row, column, mask, maximum):
```

Source: complete `gui.py` listing in Part 11, lines 114–136.

1. Build clean AES and round-eight faulty timelines, align matching stages and add the DFA equation explanation. Generate distinct masks and call the round-eight recovery adapter.
2. Add candidate-history snapshots, reverse a unique K10 and verify the master key. Return a lesson report; this retained round-eight lesson is distinct from the round-nine path used by the current Fault analysis selector.

13.2.13 `pdf_aes_lesson`

```
def pdf_aes_lesson(block, key, row, column, mask, maximum):
```

Source: complete `gui.py` listing in Part 11, lines 139–177.

1. Build a clean AES trace and process columns beginning at the selected column. For each, inject round-nine faults, show the first fault's propagation and intersect correlated key quartets.
2. Put unique quartet bytes into K10, reverse the schedule and verify the master key. If a column remains ambiguous, return an explicit incomplete report and stop rather than completing it from the teaching key.

13.2.14 sm4_master_lesson

```
def sm4_master_lesson(block, key, start_byte, mask, maximum, giraud_mode=False, known_bit=False):
```

Source: complete `gui.py` listing in Part 11, lines 180–208.

1. Record a clean SM4 lesson and define an output-only oracle plus an observer for candidate/peeling events. Choose the controlled-byte engine or the Giraud-inspired wrapper from giraud_mode.

2. Build snapshots from clean/faulty peeled views, verify a complete master key and append a recovery or ambiguity result. Return subkeys, history, counts and display metadata; no reference subkeys are supplied to the solver.

13.2.15 sm4_master_lesson.oracle

```
def oracle(number, byte, value):
```

Source: complete `gui.py` listing in Part 11, lines 187–189.

1. Choose the controlled-byte or single-bit target adapter from the lesson mode, capture the simulation block/key and return only the requested faulty output. Do not expose trace dictionaries to the solver.

13.2.16 sm4_master_lesson.show

```
def show(event):
```

Source: complete `gui.py` listing in Part 11, lines 191–193.

1. Convert each full-recovery event into a teaching snapshot comparing faulty_view with clean_view. Put raw outputs, already recovered later keys, inverse-L difference and candidate survivors in the explanation details.

13.2.17 pdf_sm4_lesson

```
def pdf_sm4_lesson(block, key, start_byte, mask, maximum):
```

Source: complete `gui.py` listing in Part 11, lines 211–213.

1. Accept the block, key and selected byte/mask/observation settings.

2. Call sm4_master_lesson with the default controlled-byte model. Its name retains the connection to the PDF but the recovery extends the last-round example to four target rounds.

13.2.18 aes_giraud_encrypt

```
def aes_giraud_encrypt(plaintext, key, byte=0, mask=None):
```

Source: complete `gui.py` listing in Part 11, lines 218–220.

1. Accept a target block/key and optional AES byte/one-bit mask.

2. Delegate to giraud.aes_fault_encrypt and return its output and final-round trace. This adapter has no independent AES or candidate implementation.

13.2.19 giraud_lesson

```
def giraud_lesson(algorithm, plaintext, key, row=0, column=0, mask=1, maximum=8, known_bit=False):
```

Source: complete `gui.py` listing in Part 11, lines 223–281.

1. Validate algorithm and coordinates and generate distinct one-bit masks. Route SM4 to sm4_master_lesson; for AES, visit every target byte beginning at the selected coordinate and intersect candidate bytes at their observed final output positions.
2. Add first-fault stage explanations and per-observation survivor sets. Once all sixteen AES K10 bytes are unique, reverse and verify the schedule; otherwise preserve an explicit ambiguous result.

13.2.20 CryptoLab.__init__

```
def __init__(self, root):
```

Source: complete `gui.py` listing in Part 11, lines 285–312.

1. Create application state, report/index/timer fields, a thread-safe queue and Tk variables for inputs, settings and status. Configure the window theme and call build.
2. Connect selection changes, keyboard navigation, close handling and periodic queue polling. No cipher is run merely by constructing the window.

13.2.21 CryptoLab.label

```
def label(self, parent, text, size=11, color=TEXT):
```

Source: complete `gui.py` listing in Part 11, lines 314–315.

1. Accept a parent, text, optional size and color.
2. Construct and return a consistently styled Tk Label using the parent's background. Layout is controlled by the caller.

13.2.22 CryptoLab.button

```
def button(self, parent, title, action, primary=False):
```

Source: complete `gui.py` listing in Part 11, lines 317–318.

1. Accept a parent, label, callback and primary-style flag.
2. Return a styled Tk Button bound to the callback. Constructing it does not invoke the action.

13.2.23 CryptoLab.build

```
def build(self):
```

Source: complete `gui.py` listing in Part 11, lines 320–434.

1. Construct the sidebar controls for algorithm, lesson, key, block and fault settings, then create the explorer and result tabs.
2. Allocate the three byte matrices, details box, playback/navigation controls and export action, retaining widget handles on self. Initialize disabled navigation until a lesson is available.

13.2.24 CryptoLab.selection_changed

```
def selection_changed(self, *_):
```

Source: complete `gui.py` listing in Part 11, lines 436–452.

1. Ignore selector callbacks while a computation is busy. Determine whether the chosen mode is a fault lesson, Giraud lesson or SM4 and update labels and explanatory hints.
2. Enable only relevant controls: SM4 has no AES column, and exact-bit knowledge applies only to Giraud. Keep any existing report tied to its original inputs and warn that rebuilding is needed to apply new settings.

13.2.25 CryptoLab.load_example

```
def load_example(self):
```

Source: complete `gui.py` listing in Part 11, lines 454–460.

1. If not busy, choose the AES or SM4 official key/input pair.
2. Load ciphertext instead of plaintext when Decrypt is selected, then update status. This changes controls only; it does not build a new timeline automatically.

13.2.26 CryptoLab.set_text

```
def set_text(self, widget, text):
```

Source: complete `gui.py` listing in Part 11, lines 462–466.

1. Temporarily enable a read-only text widget, clear its content and insert the supplied text.
2. Disable the widget again. This centralizes safe updates for details and summary boxes.

13.2.27 CryptoLab.run

```
def run(self):
```

Source: complete `gui.py` listing in Part 11, lines 468–493.

1. Ignore a duplicate request while busy, stop playback, parse input fields and validate attack coordinates, masks and observation limits. Report invalid input without starting a worker.
2. Capture settings, disable editing and start a daemon thread running compute. The captured arguments prevent later widget changes from silently altering the running experiment.

13.2.28 CryptoLab.compute

```
def compute(self, algorithm, operation, key, block, row, column, mask, maximum, known_bit=False):
```

Source: complete `gui.py` listing in Part 11, lines 495–512.

1. On the worker thread, choose the Giraud, controlled-fault or normal trace builder. For ordinary encrypt/decrypt, run the inverse and check the original block is recovered.
2. Add the original settings to the report and put either the report or an error string into the queue. Do not update Tk widgets from this worker thread.

13.2.29 CryptoLab.poll

```
def poll(self):
```

Source: complete `gui.py` listing in Part 11, lines 514–539.

1. On the Tk thread, try to consume a queued report or error without blocking. When a result arrives, release busy state and restore the appropriate controls.

2. Display errors or install the new report, step list, matrices and summary; enable export and refresh navigation. Schedule another poll with root.after so the event loop remains responsive.

13.2.30 CryptoLab.render

```
def render(self):
```

Source: complete `gui.py` listing in Part 11, lines 541–565.

1. Take the selected snapshot and choose its explicit correct reference or the previous displayed state. Decode both blocks, compute XOR and changed-byte/bit counts, and choose AES column-major or SM4 word-row layout.

2. Update titles, three matrices, red highlights, detail text and navigation indicators. Changing the displayed step never reruns encryption or changes solver results.

13.2.31 CryptoLab.navigation

```
def navigation(self):
```

Source: complete `gui.py` listing in Part 11, lines 567–573.

1. Determine whether a report is available and computation is idle.

2. Enable or disable Previous, Next, Play, step selection and the slider based on this state and current index. This guards navigation at the timeline boundaries.

13.2.32 CryptoLab.select

```
def select(self, index):
```

Source: complete `gui.py` listing in Part 11, lines 575–579.

1. If a report is ready and the application is idle, stop automatic playback.

2. Store the supplied valid step index and call render. Internal callers obtain indices from the slider, dropdown or bounded movement methods.

13.2.33 CryptoLab.seek_to

```
def seek_to(self, value):
```

Source: complete `gui.py` listing in Part 11, lines 581–584.

1. Convert the slider's textual numeric value to an integer step index.

2. If it differs from the displayed index and a report exists, delegate to select. Avoiding redundant selection also prevents unnecessary re-rendering.

13.2.34 CryptoLab.move

```
def move(self, direction):
```

Source: complete `gui.py` listing in Part 11, lines 586–588.

1. Require an existing report and add the requested direction to the current index.
2. Clamp the result to the first and last valid indices and call select. Previous/Next and keyboard actions share this boundary handling.

13.2.35 CryptoLab.play

```
def play(self):
```

Source: complete `gui.py` listing in Part 11, lines 590–600.

1. If a timer is already scheduled, stop playback; otherwise require an idle application and report. Restart from the first step when already at the end.
2. Set the Pause label, render and schedule tick after the configured millisecond delay. Tk's event loop remains in control rather than being blocked by a sleep loop.

13.2.36 CryptoLab.tick

```
def tick(self):
```

Source: complete `gui.py` listing in Part 11, lines 602–609.

1. Clear the stored timer identifier and advance by one timeline step.
2. Render the new step, schedule another tick when more steps remain, or stop at the end. This method advances a precomputed lesson rather than executing a cipher round.

13.2.37 CryptoLab.stop

```
def stop(self):
```

Source: complete `gui.py` listing in Part 11, lines 611–615.

1. Cancel a scheduled after callback if one exists and clear its identifier.
2. Restore the Play label. It is safe to call this while no timer is active.

13.2.38 CryptoLab.fill_summary

```
def fill_summary(self):
```

Source: complete `gui.py` listing in Part 11, lines 617–633.

1. Read the installed report and assemble input/output, model and verification information. For fault lessons, include used observations, recovered key/subkeys and ambiguity or bit-knowledge details.
2. For normal execution, include inverse verification and recovered input. Write the text through set_text, explicitly distinguishing simulation data and teaching-key export from a physical experiment.

13.2.39 `CryptoLab.export`

```
def export(self):
```

Source: complete `gui.py` listing in Part 11, lines 635–645.

1. Require a report and ask the user for a JSON filename.
2. Serialize the complete current report with indentation and update status; catch file-writing errors in a dialog. The export includes the teaching key and hidden snapshots, so it is a lesson archive rather than an attacker-only observation file.

13.2.40 `CryptoLab.close`

```
def close(self):
```

Source: complete `gui.py` listing in Part 11, lines 647–650.

1. Stop playback and cancel periodic queue polling.
2. Destroy the root window. No cryptographic key recovery is performed during shutdown.

13.2.41 Anonymous event handlers and the worker lifecycle

The lambdas created in `build` connect buttons to movement methods, preserving their direction argument. Keyboard lambdas suppress navigation when focus is in a text entry or dropdown. The step-selector handler reads the chosen list index; the speed control supplies a millisecond delay, not the number of cipher rounds. These expressions return presentation actions rather than candidate bytes.

The complete execution order after clicking Build lesson is:

`run` ⟶ `compute (worker)` ⟶ `messages queue` ⟶ `poll (Tk thread)`.

After installing the report, `render` displays a snapshot and `fill_summary` displays the result. Playback follows `play`, repeated `tick`, and `stop`; export follows `export` and writes this already computed report. Neither playback nor export repeats a fault experiment.

13.3 Return values, mutation and error handling at a glance

Normal cipher outputs: AES and SM4 block functions return sixteen immutable bytes. They do not print, pause, add message padding, choose an encryption mode or authenticate a message.

Trace outputs: target adapters return ciphertext and a trace; GUI trace functions return output and snapshot dictionaries. These teacher views may contain internal states. Solver calls receive a deliberately smaller interface.

Candidate outputs: AES Piret returns a set of four-byte tuples; AES Giraud and SM4 filters return byte candidate sets and position/model diagnostics. A tuple correlation cannot be replaced by choosing four independent byte survivors.

Recovery outputs: complete keys are returned only after uniqueness. SM4 reports include `complete`, `master_key`, `round_keys`, `history`, `candidate_counts` and `used`. Incomplete recovery returns no master key, even if some subkeys are already known.

Errors: invalid sizes/masks and incompatible differential support raise `ValueError`; failed verification raises `RuntimeError`. A compatible but nonunique intersection is reported as ambiguity rather than being mislabeled an exception or a successful key guess.

References and Scope

1. Student-provided handwritten notes, *AES/SM4 notes*, 27 pages, including AES fundamentals, Piret–Quisquater, SM4 fundamentals, and an SM4 DFA example.
2. NIST, *FIPS 197: Advanced Encryption Standard (AES)* (algorithm and official AES test vector).
3. ISO/IEC 18033-3, *Encryption algorithms — Part 3: Block ciphers* (SM4 specification); SM4 test vector as reproduced in the supplied notes.

Note. The SM4 DFA chapter uses a deliberately strong fault assumption: known XOR magnitude at a specified S-box input. Experimental fault injection must establish whether that assumption is justified. The included Python code demonstrates full recovery with a simulated multi-round fault oracle; no full physical recovery of all four SM4 final round keys is claimed here.